

Fine-Grained Dynamic Update Controller Synthesis

Supplementary Proofs and Tables

Anonymous Authors

This supplement uses the finite contract, four successor rules, and algorithm in the main paper. Its proofs use the confirmed non-preemptive rule: update events have no successor whenever a modeled uncontrollable event is enabled. Goals are safe, uncontrollable-quiescent states with all update events completed and a compatible supplied endpoint. No fairness assumption is added. Section S4 reports completed correctness/capability and Xeon results, retaining unresolved outcomes and original trial provenance.

S1 Granularity and observation witnesses

The main paper gives the two completing paths and the local-versus-bulk argument. Here we give the complete finite contract, tabulate its full closure, define the additional observation and requirement-boundary witnesses, and give their complete proofs. We also explain how the smaller executable fixtures relate to these contracts.

S1.1 Complete finite contract and paths of the cell

Table S1 gives the finite LTSs: o/n mark versions, h/e holding/empty, m_A, m_B move the product to A, B , and j inspects at B . Monitor states k, c, d, err mean old-safe, inspection-clear, inspection-pending and error; c_0, c_1 are distinct controller states. All events are controllable. Ordinary events are $\{m_A, m_B, j\}$; the updates are ρ_A, ρ_B , start, stop, where $\text{start} := \text{start}(r_{\text{inspect}})$, $\text{stop} := \text{stop}(r_{\text{old}})$, and $\text{start} \prec \text{stop}$.

Table S1: Cell components, controllers and monitors. Parentheses mark initial states.

Object	States (initial)	Ordinary transitions
A^o, A^n	$h, e (h)$	$h \xrightarrow{m_B} e, e \xrightarrow{m_A} h$
B^o	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e$
B^n	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e, h \xrightarrow{j} h$
C^o	$c_o (c_o)$	$c_o \xrightarrow{m_A, m_B} c_o$
C^n	$c_0, c_1 (c_0)$	$c_0 \xrightarrow{m_A, j} c_0, c_0 \xrightarrow{m_B} c_1, c_1 \xrightarrow{j} c_0, c_1 \xrightarrow{m_B} c_1$
$t_{r_{\text{one}}}$	$A, B, \text{err} (A)$	$A \xrightarrow{m_B} B, B \xrightarrow{m_A} A, A \xrightarrow{m_A} \text{err}, B \xrightarrow{m_B} \text{err}$
$t_{r_{\text{old}}}$	$k, \text{err} (k)$	$k \xrightarrow{j} \text{err}$
$t_{r_{\text{inspect}}}$	$c, d, \text{err} (c)$	$c \xrightarrow{m_B} d, d \xrightarrow{j} c, d \xrightarrow{m_A} \text{err}$

Transfers are $g_A = g_B = \{(e, e)\}$, defined only at empty stations. Old/new/interval requirements are $r_{\text{old}}/r_{\text{inspect}}/r_{\text{one}}$. The interval initializer records the holder at either all-old one-product tuple. The new initializer accepts all eight version-tagged one-product tuples, installing d if B holds, c otherwise (the main paper's requirement interpretation).

Endpoint tuples list A, B , controller and endpoint-monitor states in that order. The reachable old tuples are $x_A = (h, e, c_o, k)$ and $x_B = (e, h, c_o, k)$; the new tuples are $z_A = (h, e, c_0, c)$, $y_B = (e, h, c_1, d)$ and $z_B = (e, h, c_0, c)$. Initial tuples are x_A, z_A ; the loadable targets are $Z_{\text{load}} = \{z_A, z_B\}$,

Table S2: One policy from both entries. Each tuple is (versions of A, B ; holder; old/new monitors); the holder also determines the interval monitor. A dash means inactive. Each column block is one path; successive rows give its six or five edges and remaining-step rank.

A -holding entry			B -holding entry		
Incoming step	State	Rank	Incoming step	State	Rank
entry	$(oo; A; k-)$	6	entry	$(oo; B; k-)$	5
ρ_B	$(on; A; k-)$	5	ρ_A	$(no; B; k-)$	4
m_B	$(on; B; k-)$	4	m_A	$(no; A; k-)$	3
start	$(on; B; kd)$	3	start	$(no; A; kc)$	2
stop	$(on; B; -d)$	2	stop	$(no; A; -c)$	1
j	$(on; B; -c)$	1	ρ_B	$(nn; A; -c)$	0
ρ_A	$(nn; B; -c)$	0			

and both reachable old tuples must admit an update. Handover excludes y_B : inspection must be clear at completion.

Component and controller alphabets contain exactly their displayed outgoing labels. Unlisted component or controller transitions are absent; unlisted non-error monitor transitions self-loop, and err absorbs every label. Comma-separated labels are separate edges. Transfer changes one empty station to its new version; start installs the new monitor and stop removes the old one. Other active monitors observe these events. The policy selects $\rho_B; m_B; \text{start}; \text{stop}; j; \rho_A$ from x_A and $\rho_A; m_A; \text{start}; \text{stop}; \rho_B$ from x_B , and is empty elsewhere. These two paths determine 13 states and 11 edges. Their last states map to z_B, z_A , respectively; S1 also displays the states.

Local versus simultaneous transfer. Both entries have safe singleton-outcome paths without uncontrollables. The x_A path is the sequence in the upper path above. The remaining lengths $6, \dots, 0$ and $5, \dots, 0$ strictly decrease to the compatible targets z_B, z_A , proving finite completion.

For simultaneous transfer, replace the two commands by one whose relation is $g_A \times g_B$, preserving the endpoints and requirements over the same cell states. It requires both stations empty. Ordinary moves preserve exactly one product, inspection preserves its position, and requirement boundaries leave it unchanged. The joint transfer is therefore never enabled; it stays pending, making every goal unreachable. The comparison holds endpoints and requirements fixed.

S1.2 Embedding and the Two-Station Cell

For a global-transfer input with a single component $\star$, identify its old/new component states with the corresponding old/new product states. Carry monitor states and pending requirement operations unchanged. The map on update states is a bijection. The ordinary rule synchronizes the same transitions, the transfer rule applies the same global relation, and requirement stop/start applies the same monitor update and initialization. The sets of modeled uncontrollables coincide, so the non-preemption guard also coincides. Safety, roots, target matching, and quiescence are preserved. This establishes the game isomorphism for the global-transfer fragment.

For $\mathcal{W}_{\text{cell}}$, every ordinary move preserves exactly one product. Transfer changes only an empty station's version, and inspection leaves the product in place. Consequently the holder is always A or B . Using the main paper's cell-state notation and abbreviating start/stop by s/t , the two retained paths are

$$\begin{aligned}
 p_A : \quad & ooA; k- \xrightarrow{\rho_B} onA; k- \xrightarrow{m_B} onB; k- \xrightarrow{s} onB; kd \xrightarrow{t} onB; -d \xrightarrow{j} onB; -c \xrightarrow{\rho_A} nnB; -c, \\
 p_B : \quad & ooB; k- \xrightarrow{\rho_A} noB; k- \xrightarrow{m_A} noA; k- \xrightarrow{s} noA; kc \xrightarrow{t} noA; -c \xrightarrow{\rho_B} nnA; -c.
 \end{aligned}$$

Every event on these paths is controllable and singleton-outcome. Old requirement is stopped before inspection; the inspection monitor is initialized to d when B holds and to c when A holds. Inspection

clears the former obligation. Thus every retained state is safe, and the two paths have disjoint roots and no contradictory action assignment. Assign remaining-path lengths $6, \dots, 0$ and $5, \dots, 0$. Their final component/new-monitor projections match z_B, z_A , respectively. These states are quiescent since the fixture has no uncontrollables. This proves all-root winning, not just a path from the initial old component tuple.

For the bulk variant, $g_A \times g_B$ is defined only if both stations are empty. No ordinary transition, requirement event, or available transfer can create that state from a one-product root. Hence ρ_* stays pending in every reachable state, and there is no goal. Under any policy, a maximal execution either ends outside a goal or continues indefinitely; both violate finite completion. This obstruction is independent of the ordinary moves that remain available.

Table S3: Full reachable closure of the paper’s illustrative two-station fixture, including successors beyond unsafe states. This census is distinct from terminalized solver work.

Quantity	Local transfer	Bulk transfer
Reachable states	56	10
Safe / unsafe states	26 / 30	6 / 4
Goals	2	0
Candidate buckets (all labels)	392	60
Singleton / empty buckets	139 / 253	16 / 44
Retained policy states / edges	13 / 11	–

Local counts are 30 for m_A , 26 for m_B , 20 for j , 15 for ρ_A , 10 for ρ_B , 12 for s , and 26 for t . Twelve safe-to-unsafe buckets remain visible: eight m_A edges violate inspection and four j edges violate the still-active old requirement. The holder monitor never reaches error. Bulk counts are 5 for each move, 2 for s , 4 for t , and zero for inspection and transfer. The primitives and omitted-transition conventions determine this census; these counts are not additional rules.

S1.3 Observation-dependent choice

In $\mathcal{W}_b$, all five old component states are reachable under the permissive endpoint controller. The policy waits at w , permits a at a_0 and b at b_0 , and transfers at a_1, b_1 . Both outcomes of u are retained. Ranks 3, 2, 1, 0 by distance to the compatible new endpoint certify all five roots.

Consider any fixed finite list of controllable events, with uncontrollables accepted without advancing the list position. The environment can take u at w before the next listed controllable. Its two outcomes leave the same list position at a_0 and b_0 . Any finite sequence of *idle* heads changes neither component state; an infinite sequence would itself fail completion. The next non-idle head is one of a, b, d_A, d_B, ρ . At a_0 , only a among these is enabled; at b_0 , only b is enabled. Thus no common head can complete both outcomes. An exhausted list also fails at these non-goals. This proves failure for every such fixed list from the root w , and therefore for all-root admission.

The executable branching fixture tests the unrestricted policy and two all-time safety restrictions, $\Box \neg b$ and $\Box \neg a$. Each restriction has a losing branch and hence gives one of the two observed LOSS jobs. These restrictions are concrete obstruction checks; the universal statement about fixed lists follows from the head argument above. In the neutral fixture, both resolving events are available after each branch, so the unrestricted and both restricted tests win.

S1.4 Requirement interleaving

The old endpoint of $\mathcal{W}_r$ stays at w using *idle*; its controller disables the controllable *align* to satisfy the old monitor. The new endpoint at n is likewise safe and deadlock-free. Update admission drops the old controller while retaining the old monitor. The sequence

$$\text{stop}(r_o); \text{align}; \text{start}(r_n); \rho$$

first deactivates the old prohibition, reaches the transfer-ready state, starts the new prohibition from its supplied safe residual, and transfers. Every retained state is safe, and the final state matches the only load target.

Every completion must execute ρ , which requires the old ready state. Only *align* reaches that state. While r_o is active, *align* reaches its error; once r_n is active, *align* reaches its error. Therefore a safe completion must place *align* strictly between stop and start. A policy requiring the requirement operations to form a contiguous block excludes this necessary event. Simultaneous stop/start excludes the same gap. Starting first cannot help: it is disallowed at w by $\text{dom } \iota_{r_n}$, and overlap of the prohibitions would still forbid *align*.

The executable separation fixture is a related test input, not the same transition system as $\mathcal{W}_r$. It requires $\text{startNewSpec} \Rightarrow \text{Aligned}$ with persistent *Aligned* set by *align* and reset at entry. Its NEW monitor instead uses the frontend's current-fluent lookup: the forbidden-event observer *align_a* must be false at activation. An ordinary *align* sets both *Aligned* and *align_a*; *idle* clears *align_a* while preserving *Aligned*, and transfer/lifecycle events preserve these observer values. Consequently *stop; align; idle; \rho; start* is a source-derived five-step winning witness. The saved separation WIN has six certificate states and maximum rank five; the logs contain only transition summaries, so they do not establish the returned policy's exact placement of transfer. The four-step witness above belongs only to the abstract $\mathcal{W}_r$.

Both executable inputs retain the interval requirement that persistent alignment hold when new requirement starts. Neutral is not removal of this requirement. Both actual separation endpoints have one state and only *idle* loops. In the actual neutral model, the old endpoint has one *idle* state; the new endpoint has two states, $\text{NEW_WAIT} \xrightarrow{\text{align}} \text{NEW_READY}$, with *idle* at both. Its OLD/NEW monitors forbid *bad*, and NEW activation requires $\text{bad}_a = \text{false}$. It therefore permits transfer and alignment before the requirement block. The neutral full/block certificates have maximum ranks five/four; the full policy's extra step is not identified from the summary. The executable block forbids transfer as well as ordinary events between requirement operations. Its separation obstruction still follows because old requirement prevents alignment before stop and the interval requirement prevents starting before alignment. No trace equivalence or monitor-state isomorphism between the abstract and executable encodings is claimed.

S1.5 Complete parameters for the smaller witnesses

$\mathcal{W}_b$. $\Sigma_N = \{\text{idle}, u, a, b, d_A, d_B\}$; only u is uncontrollable. $g = \{(a_1, x), (b_1, y)\}$, $\Sigma_{\text{upd}} = \{\rho\}$, $\prec = \emptyset$. One-state C^o/C^n loop on all six ordinary labels/ $\{\text{idle}, d_A, d_B\}$, respectively. All requirement, monitor and ι families are empty. $Q_0 = \{((o, s), (), \{\rho\}) : s \in \{w, a_0, b_0, a_1, b_1\}\}$; $Z_{\text{load}} = \{(x, c_n), (y, c_n)\}$. Goals are the two new states with empty P ; κ appends c_n .

$\mathcal{W}_r$. All events controllable; $\Sigma_N = \{\text{idle}, \text{align}\}$, $g = \{(r, n)\}$, $\Sigma_{\text{upd}} = \{\rho, t, s\}$, $t = \text{stop}(r_o)$, $s = \text{start}(r_n)$, $\prec = \emptyset$. One-state C^o has alphabet $\{\text{idle}, \text{align}\}$ but only an *idle* loop; C^n has only an *idle* loop. $R^{\text{old/new/upd}} = \{r_o\}/\{r_n\}/\emptyset$; $\text{dom } \iota_{r_n} = \{(o, r), (n, n)\}$, $\iota_{r_n} = k$. Its supplied residual forbids *align* after activation and erases ordinary events before activation (exact RS). $\text{Reach}(\text{CL}^{\text{old}}) = \{(w, c_o, k)\}$, $Q_0 = \{((o, w), (k, \perp), \{\rho, t, s\})\}$; $Z_{\text{load}} = \text{Reach}(\text{CL}^{\text{new}}) = \{(n, c_n, k)\}$. The unique goal is $((n, n), (\perp, k), \emptyset)$; κ appends c_n .

Arrows entering a state mark local initials; dashed edges denote transfer. Unlisted plant/controller transitions are empty, unlisted non-error monitor transitions self-loop, and E absorbs all labels. Every endpoint admits *idle* and is deadlock-free.

S2 Finite games, search invariants, and certificates

Let $R = \text{Reach}_\sigma(Q_0)$, computed without expanding goals. A maximal run is either infinite, or finite and ending at a goal or at a state with no policy successor. All statements below concern reachable states; a cycle outside R is irrelevant to a particular policy.

S2.1 Complete eligibility checks

Before synthesis, we check consistency of event ownership and controllability; monitor totality, determinism, and absorbing errors; the types of g_i and ι_r ; for each $r \in R^{\text{upd}}$, membership in $\text{dom } \iota_r$ of the pre-update-tagged component state of every $z \in \text{Reach}(\text{CL}^{\text{old}})$; that $\prec$ is a strict partial order; requirement satisfaction and deadlock freedom of the pre-update and post-update closed loops; inclusion of every environment event in the event set of the corresponding controller; non-suppression of uncontrollable events by the pre-update and post-update controllers; and reachability of Z_{load} . RS remains external evidence. Synthesis checks ι_r 's type, domain and non-error property; completeness of its activation histories is supplied, rather than inferred.

S2.2 Maximal runs and ranks

Assume R is finite, all its states are safe, goals are terminal, and every non-goal has a successor. If the reachable non-goal subgraph has a cycle, first follow a path from a root to that cycle and then repeat it. Every edge is policy-permitted, including its selected nondeterministic outcome, so this is an infinite maximal run avoiding goals. Universal finite completion excludes it without invoking fairness.

Conversely, if that subgraph is acyclic, no path can visit a non-goal twice. Every path therefore has at most $|R \setminus Q_{\text{goal}}|$ non-goal edges before ending. A non-goal endpoint is impossible by the successor premise. Thus every maximal run ends at a goal. Define $\text{rk}(q)$ as the maximum length of a goal-ending path from q , and set goal ranks to zero. Such a path exists because the graph is finite, acyclic off goals, and has no non-goal sinks. For each edge $q \rightarrow q'$ from a non-goal, prefixing a longest path from q' proves $\text{rk}(q) \geq 1 + \text{rk}(q')$. This gives a positive natural-number rank at non-goals. Finally, a strict decrease around a cycle would give $\text{rk}(q) > \text{rk}(q)$, a contradiction. These implications establish all three equivalences. Without the successor premise, a non-goal sink is an acyclic counterexample.

S2.3 The four exploration invariants

For the partial table $\mathcal{D}$, queried-empty buckets are recorded by candidate consumption even though they are absent from $\text{dom } \mathcal{D}$. Each stored nonempty bucket is exact and contains all modeled outcomes. We state invariants at loop boundaries, after discovery queues and the finite reverse-edge worklist have been updated.

1. **Uncontrollable completeness.** On first expansion of q , all uncontrollable candidates are consumed before promotion can use the state. No candidate is added later: the finite event set is determined by q . Therefore every $q \in X$ has its complete uncontrollable successor set known. The transient insertion into X inside the expansion block is not a return point.
2. **Controllable exhaustion at failure.** A safe non-goal state with no uncontrollable has a cursor over all controllable candidates. Expansion consumes it to the next nonempty bucket or exhaustion. If a nonwinner has candidates remaining, it is in Retry. Each retry consumes further candidates. The failure return is reached only after both queues and the propagation worklist are empty. Hence every such state in $V \setminus W$ has exhausted its cursor. Removing an entry when its state becomes winning cannot remove a losing candidate from this argument.
3. **Monotone winning evidence.** A promoted state either has all uncontrollable outcomes at smaller retained ranks, or has a complete selected controllable bucket at smaller ranks and no uncontrollable. All those buckets are exact; growing $\mathcal{D}$ adds other controllable choices rather than new outcomes to an already stored choice. The policy disables those additional controllables. Existing rank-decreasing witness edges consequently remain a valid certificate. Goals are recognized using modeled uncontrollable enabledness before terminalization, so an unqueried uncontrollable cannot invalidate rank zero.

4. **Open-state coverage.** Each newly discovered safe non-goal is inserted once in *Open*. It is removed only for first expansion, when it enters X . Thus $V \cap \text{Safe} \setminus (X \cup Q_{\text{goal}}) \subseteq \text{Open}$. Unsafe states and goals are discovered terminals and are handled separately.

The initial root insertion establishes these properties. Expansion, successor discovery, candidate consumption, and promotion preserve them as shown above; no other operation changes their relevant sets.

Fixed exploration order in the measured implementation. The RQ3 LTS configurations disable the preliminary guided search (both guided limits are zero). The complete algorithm queries every uncontrollable candidate first. If any such bucket is nonempty, it omits controllable queries by the UC-priority reduction. The underlying Post relation still contains any enabled ordinary controllable edges, which Direct-Full materializes. Otherwise lazy and update-first consume their sorted controllable cursor through the first nonempty bucket, deferring the rest; eager consumes every controllable candidate. A deferred candidate is never removed merely because it appears unpromising.

The default lazy/eager priority depends on the pending operations and endpoint projection, as follows. Entries within each row have increasing priority; all listed types remain candidates if the game enables them.

Condition	Priority classes (first to last)
Some old stop pending	stop; transfer; ordinary; other update
No stop, some transfer pending	transfer; ordinary; other update
No stop or transfer, endpoint aligned	start; all other events
No stop or transfer, not aligned	events other than start; start
Update-first ablation	every update; every ordinary event

Within a class, the implementation prefers updates if update-first is configured or the endpoint projection is aligned, and ordinary events otherwise; remaining ties use type order start, transfer, stop, ordinary, then Java string order on the event label. Endpoint mismatch is the minimum over load signatures of the number of disagreeing component/new-monitor coordinates. An old coordinate agrees when its transfer contains the target; an inactive new monitor agrees when its defined activation map produces the target state.

Open is a LIFO stack. Initial roots and each newly discovered batch are sorted by increasing endpoint mismatch, then pending-set size, then canonical state order, and inserted in reverse so the least element is popped first. Existing stack entries keep their positions; this is not a global priority queue. Canonical ties compare component count, then each component's version and state, sorted active-monitor identifiers and states, and sorted pending labels; the problem's comparators define local state order. Already queued/expanded states are not inserted again. Retry is FIFO: append a state when it first has deferred candidates, remove from the head, skip winners/exhausted cursors, consume through the next nonempty bucket, and append again if candidates remain. Drain propagation after each expansion/resumption, and always serve *Open* before *Retry*. These rules are implemented by `FineGrainedSuccessorOracle` and `OtfDuccSynthesizer`; the artifact retains their exact source. Eager changes bucket materialization and update-first changes candidate priority, providing the reported sensitivity checks without selecting an order after observing results.

S2.4 Attractor, termination, soundness, and completeness

Let $W_0 = V \cap Q_{\text{goal}}$. At the next layer a safe expanded state joins if its nonempty uncontrollable union is contained in the preceding layers, or if it has no uncontrollable and one complete nonempty controllable bucket contained there. Induction on these layers proves the existence of a winning policy for the current table. These minimum layer indices are analytical, not the implementation's retained certificate ranks. On first promotion the implementation assigns one plus the largest rank of the complete chosen successor set (all uncontrollable outcomes, or one winning controllable choice); goals have rank zero. It

never lowers that assigned rank. Every stored choice used for promotion remains complete and rank-decreasing. Extraction may choose another stored controllable set only if all its successors also have smaller retained ranks. Goals satisfy the target and quiescence checks. Invariant I3 therefore preserves a valid certificate as exploration continues without claiming rank minimality.

For termination, there are finitely many reachable states and state/event candidates. Each state is discovered and expanded at most once, and each cursor position is consumed at most once. Retry always consumes a new candidate, and worklist propagation inserts a state into W at most once. After finite work the algorithm either covers every root or empties its queues. This argument needs no fairness of the modeled environment; it is an argument about the finite synthesis procedure.

If every root is in W , the induced policy graph is safe and closed, has a nonempty successor set at every non-goal, and has a strict rank. The maximal-run lemma proves finite completion. Goal recognition provides a compatible $\kappa(q)$. Section S3 supplies the monitor-interval induction and the relative requirement guarantee.

If the algorithm returns LOSS, let $L = V \setminus W$. Invariant I4 ensures that each safe non-goal in L was expanded, and I1 provides all uncontrollable outcomes. If an uncontrollable is enabled and all its outcomes across all uncontrollables were in W , the state would have been promoted. Hence some uncontrollable outcome lies in L . When there is no uncontrollable, I2 supplies every controllable bucket. If a nonempty bucket were contained in W , the state would likewise be promoted. Therefore every enabled controllable has an outcome in L . The failure return leaves a root in L and excludes goals from L .

Against any policy at an uncontrollable state, the environment selects the exhibited uncontrollable outcome in L . Otherwise, it selects a remaining outcome of whichever enabled controllable the policy permits; if none is permitted, there is a non-goal deadlock. Repeating yields an unsafe state, a non-goal deadlock, or an infinite run in L . Each violates winning. Thus failure is sound for all policies, including policies enabling several controllables. Conversely, a winning policy rules out such a failure. Together with termination, this proves completeness for the fixed successor and goal rules. Equivalently, a winning policy can be normalized by removing all controllables in the presence of an uncontrollable, and retaining one winning controllable elsewhere: only runs are removed and non-goal successors remain nonempty.

S2.5 Checking the returned evidence

A WIN checker reconstructs the reachable policy domain from all roots and complete Post buckets. It verifies safety, enabledness of every retained action, nonempty successors outside goals, closure in the rank domain, and strict rank decrease for every outcome. At each goal it checks rank zero, physical uncontrollable quiescence, membership of $\kappa(q)$ in Z_{load} , and component/new-monitor equality. Omitted policy entries mean the empty set. The maximal-run lemma then proves the model-level claim without reproducing the synthesizer's exploration order.

A LOSS checker verifies a nonempty intersection with the roots, no goal in L , and the obstruction conditions just proved. Unsafe states are terminal losses. At a safe uncontrollable-free deadlock the universal condition on enabled controllables is vacuous, correctly certifying a loss. A safe uncontrollable-free state with choices requires inspecting every enabled candidate, so checking LOSS may cost as much as the corresponding exhaustive search. These checks trust the shared parser, successor, monitor, transfer, and endpoint/goal semantics; the separately implemented explicit and raw-LTS checks in RQ1 provide additional, scoped cross-checks.

S2.6 Work bound and static encoding

After endpoint/eligibility preprocessing, put $N = |\mathcal{Q}|$, $A = \sum_q |\text{Cand}(q)|$, and $B = \sum_{q,a} |\text{Post}(q, a)|$, counting nonempty outcomes in B . With indexed constant-time state predicates, expected-amortized constant-time dictionaries, and $O(1 + |\text{Post}(q, a)|)$ successor enumeration, state discovery and expansion cost $O(N)$, consumed candidates cost $O(A)$, and stored outcomes/reverse incidences cost $O(B)$.

Each bucket maintains its number of nonwinning outcomes and maximum winning-successor rank. Its reverse incidences are visited only on a successor's first winning insertion. Each state aggregates its uncontrollable buckets. A duplicate-free queue, one-way cursor, and intrusive retry-removal handle charge each queue operation to a discovery, candidate consumption, or promotion. A remaining-root counter avoids repeated all-root scans. Certificate extraction scans its retained domain once. These charges give $O(N + A + B)$ time and space after preprocessing.

Let preprocessing cost T_{end}, S_{end} . The total bounds are $O(T_{end} + N + A + B)$ and $O(S_{end} + N + A + B)$. They are bounds on this indexed implementation model. The Java implementation sorts and scans active transitions for quiescence and repeatedly checks roots; its timings include that work and are not measurements of constant-cost semantic queries. Preprocessing cannot be bounded by the reached update game alone; all-entry sets can themselves be exponential in compact component descriptions.

For a generic reachability encoding, use a fresh environment root with edges to every Q_0 state. At a safe non-goal with uncontrollables, an environment vertex chooses any uncontrollable event and then any outcome. At an uncontrollable-free state, a controller vertex selects a nonempty bucket and an environment vertex chooses its outcome. Unsafe states and deadlocks lose, and goals win. At most one intermediate vertex per nonempty bucket and one edge per outcome are required, giving $O(N + A + B)$ construction. Normalization above proves equivalence to the supplied game; a compatible κ remains additional typed evidence.

For any root/safety/goal/controllability-preserving isomorphism preserving all buckets, including empty ones, the strong-predecessor operation commutes with the state bijection. Induction preserves every attractor layer and its rank; complementing gives corresponding losing sets. Selected actions transport through the event bijection. Transport of load targets and κ additionally requires matching typed endpoint projections; an untyped graph isomorphism alone supplies no such map.

S2.7 Full-expansion containment and the independent family

Direct-Full and OTF start from identical roots and use identical terminal tests, candidates, and successor rules. Induct on discovery by OTF. A non-root discovered state is an outcome queried at a previously discovered safe non-goal predecessor. Full expansion also discovers that predecessor by induction; since its terminal test is identical it expands the predecessor and queries the candidate, discovering every outcome. An unsafe or goal successor is recorded by both but expanded by neither. Applying this to the source of each queried bucket proves query containment as well as state containment. This reasoning covers arbitrary order and nondeterminism.

For the exact $n + 1$ versus 2^n family, component i has one old and one new state, each with a controllable *tick* loop, and its singleton transfer maps old to new. Both endpoint controllers have one state admitting *tick*; there are no requirements, uncontrollables, or dependencies. The new endpoint is the unique load target. The sole root has every transfer pending. Each update state is determined by a transferred subset $S \subseteq \{1, \dots, n\}$. It has its *tick* self-loop and, for each $i \notin S$, the transition $S \xrightarrow{\rho_i} S \cup \{i\}$. The full subset is the unique goal.

Every subset is reached by performing exactly its transfers in some order. Full expansion discovers all 2^n subsets, even though it does not expand the final goal. Under pending-transfer-first order, OTF's first nonempty bucket at each non-goal discovers one fresh subset on a chain. The open queue is served before retries, so the chain reaches the goal after n transfers. Propagation then promotes every predecessor on the chain and covers the root before any other candidate is retried. Exactly $n + 1$ states have been discovered. The *tick* self-loop at a non-goal cannot be a winning choice under universal finite completion. This is an existence construction for exploration cost, with no claim that every family or order has the same saving.

S3 Monitor intervals and conditional trace lifting

S3.1 Activation histories and residual meanings

The supplied prefix-closed SafePref_r defines safe histories. Let $H_r(\xi)$ contain the admissible activation histories under the declared requirement meaning, projected by obs_r . The history-inclusive interpretation below admits all old-loop and update-prefix histories; the alternative A and E interpretations specify different scopes. A new requirement's activation history includes its start boundary once; initialization supplies the residual after that boundary, and online monitoring does not consume the label again. Entry consumes no symbol. Define $\mathcal{C}_r(\xi) = \bigcap_{h \in H_r(\xi)} \{w : hw \in \text{SafePref}_r\}$ and $\mathcal{L}_r(m) = \{w : \eta_r^*(m, w) \notin F_r\}$. Residual-sound initialization requires

$$\mathcal{L}_r(\iota_r(\xi)) \subseteq \mathcal{C}_r(\xi) \quad (\xi \in \text{dom } \iota_r) \quad (\text{RS})$$

with equality for an exact residual and strict inclusion for a conservative one. Distinct activation histories may require finite history bits in components and transfers. RS is supplied external interpretation evidence relative to the given requirement and observation meanings. For $r \in R^{\text{upd}}$, $\text{dom } \iota_r$ includes every reachable old component state.

Actual frontend initialization. The measured frontend has two distinct paths. For each NEW safety formula, it constructs the referenced fluent observers, including implicit action fluents. Observers track their initiating/terminating events, and unrelated events stutter. A product traversal associates each reachable observer vector with a compiled safety-monitor state. Multiple non-error monitor states for one vector are accepted only if their safe-continuation languages agree; the least state identifier is retained. An error association takes precedence. Crucially, the table-building traversal does not expand error monitor states, so it does not check how observer values evolve after a violation.

`MtsaRevisedOfDucsAdapter.newActivation` selects the observer indices belonging to that NEW formula (identity first, then a unique-name match), extracts their states from the augmented component tuple, and looks up the signature. A missing signature, an unknown monitor state or an error excludes the tuple. If observer indices are empty or the mapping is absent, it falls back to the monitor's initial state. Candidate tuples are the physical closure of all old endpoint roots under ordinary events and component transfers, allowing mixed versions. $\text{dom } \iota_r$ retains exactly the non-error lookup entries; this does not prove past-history safety. The observer vector is stored with component zero, follows ordinary events in either version even while the NEW monitor is inactive, and is preserved by component transfers. At start, existing active monitors advance and the NEW monitor is installed from the lookup; it does not consume that start again.

For UPD, `compileTesters` instead computes a constant boundary state by taking the raw monitor from state zero through `hotSwapIn`, or stuttering if that label is absent. `updateActivation` assigns this same non-error state to every reachable old physical tuple; an error boundary is rejected. UPD has no fluent-based activation lookup. The symbolic contract allows both forms as supplied ι_r maps, but the fluent-determination lemma below applies to an actual initializer only after its state agrees with the history-derived state. Our coverage CSV records the initializer mode separately. Source locations (including `UpdatingControllersDefinition`, which builds the table) and extraction commands are in the artifact's `experiments/rs_coverage/README.md`.

Interpretation H: history-inclusive. For this coverage audit, obs_r retains the events defining the referenced fluents, independently of the active component version. Run the requirement monitor from m_r^0 on the whole projected history before activation and its continuation. With absorbing error states this gives

$$\mathcal{C}_r(\xi) = \bigcap_{h \in H_r(\xi)} \mathcal{L}_r(\eta_r^*(m_r^0, h)).$$

H counts violations before activation. If an admitted history reaches ERROR, its continuation language is empty; a non-error initializer accepts the empty suffix and cannot satisfy RS at that activation. We

retain the H check as a diagnostic and assert no benchmark RS guarantee under H. The guarantee sought below uses A for NEW and E for UPD; neither interpretation changes the compiled game or any saved synthesis decision. A lookup initializer uses only the current referenced fluent-observer vector $f(\xi)$, writing $\iota_r(\xi) = \sigma_r(f(\xi))$. It can forget prior violations or pending obligations. The actual new- and update-requirement frontend paths are distinguished above.

Lemma S3.1 (Fluent-determined exact initialization). *Let the freely reachable product of the deterministic requirement monitor (including absorbing errors) and all its referenced fluent observers start at their specified initial states. Suppose equal fluent vectors in that product imply equal monitor states. Let $\sigma_r(v)$ return that unique monitor state for each reachable vector v . For any activation state whose histories are represented in this product and whose supplied initializer is $\iota_r(\xi) = \sigma_r(f(\xi))$, initialization is an exact residual for the history-inclusive interpretation.*

Proof. Each projected activation history reaches a product pair $(\eta_r^*(m_r^0, h), f(\xi))$. Fluent determination makes its first coordinate equal to $\sigma_r(f(\xi))$, independently of h . Thus every language in the intersection defining $\mathcal{C}_r(\xi)$ is the same language $\mathcal{L}_r(\iota_r(\xi))$. Their intersection is that language, proving equality in RS. $\square$

A single non-error state is insufficient for this lemma. For $\Box p$ with p initially true, let a make p false and b restore it. The empty history and ab have the same current fluent value but reach the safe and absorbing-error monitor states, respectively. A check that discards errors verifies only safe-prefix determination, a weaker condition that needs a separate safe-history premise. We use the full product, including errors. Failure of this plant-independent sufficient condition leaves RS unverified for the actual activation histories; it is not by itself a counterexample to the supplied plant's RS.

Interpretation A: activation-safe NEW requirements. A new requirement becomes effective at its own activation. Its supplied meaning retains the progress or obligations represented by the current fluent vector, while excluding prior violating histories from the reference intersection. With at most one occurrence of each update event, define

$$H_r^{\text{safe}}(\xi) = \{h \in H_r(\xi) : \eta_r^*(m_r^0, h) \notin F_r\}, \quad \mathcal{C}_r^{\text{safe}}(\xi) = \bigcap_{h \in H_r^{\text{safe}}(\xi)} \mathcal{L}_r(\eta_r^*(m_r^0, h)).$$

Then RS_A is $\mathcal{L}_r(\iota_r(\xi)) \subseteq \mathcal{C}_r^{\text{safe}}(\xi)$. This is a choice of requirement meaning, not a proof that a past violating trace satisfied the original invariant. In particular, conditioning on safe histories does not validate application intent. If $H_r^{\text{safe}}(\xi)$ is empty, the intersection is the universal continuation language: inclusion is immediate, but equality generally fails. Exactness below is therefore explicitly restricted to nonempty safe-history domains.

Lemma S3.2 (A: language-determined safe-prefix initialization). *Explore the freely reachable monitor/observer product over one-shot update histories, recording but not expanding ERROR pairs. Suppose all non-error monitor states reached at the same fluent vector have equal continuation languages. On the activation domain, let the actual non-error lookup $\sigma_r(v)$ have that common language. Then $\iota_r(\xi) = \sigma_r(f(\xi))$ satisfies RS_A , with equality whenever $H_r^{\text{safe}}(\xi) \neq \emptyset$.*

Proof. Every admissible safe history reaches a non-error product pair with observer vector $f(\xi)$. By language equivalence and lookup compatibility, its residual language equals $\mathcal{L}_r(\iota_r(\xi))$. A nonempty intersection of these equal languages is that language. For an empty history set, the universal intersection contains the initialized language, proving the stated inclusion. $\square$

Language equivalence, rather than equality of state identifiers, is also the frontend's acceptance condition for multiple non-error states at a lookup vector. Our independent checker compares deterministic state pairs until an ERROR/non-error distinction or pair closure, and separately reconstructs

the frontend's error-priority, equivalent-state/minimum-ID rule. This checks the earlier saved monitor/observer exports against the source rule; those exports do not contain a materialized lookup table. A separate fresh frontend export directly checks all 138 tables (S4), while leaving physical-state decoding and actual activation histories unverified. The check establishes RS_A inclusion for all 138/138 NEW initializers without a nonemptiness premise. Equality additionally requires nonempty safe histories at the actual activation; their plant-level nonemptiness remains unverified.

Interpretation E: entry-scoped UPD requirements. An update-interval requirement becomes effective at entry. Its reference monitor starts afresh with the actual entry fluent vector v , consumes `hotSwapIn` once, and observes the subsequent update interval. Let $\mathcal{L}_r^E(v)$ denote the resulting continuation language and b_r the frontend's constant post-entry monitor state. For the entry vectors $V_r(\xi)$ compatible with ξ , define

$$\mathcal{C}_r^E(\xi) = \bigcap_{v \in V_r(\xi)} \mathcal{L}_r^E(v), \quad RS_E : \mathcal{L}_r(b_r) \subseteq \mathcal{C}_r^E(\xi).$$

The reference language must account for the effect of v on monitor initialization; simply attaching an unused observer vector to the fixed compiled DFA does not establish that relation. Before and after the one entry occurrence, each other update label is permitted at most once in the sufficient-condition overapproximation. This overapproximation may admit update labels before entry that the actual game excludes.

Lemma S3.3 (E: entry-value-independent boundary language). *Let V_r overapproximate all reachable entry observer vectors under the one-shot event constraint. If $\mathcal{L}_r^E(v) = \mathcal{L}_r(b_r)$ for every $v \in V_r$, then the constant initializer satisfies RS_E exactly at every actual entry with nonempty $V_r(\xi) \subseteq V_r$.*

Proof. Every actual entry vector belongs to V_r , so its reference post-entry language equals the boundary language by hypothesis. Thus the nonempty intersection over $V_r(\xi)$ consists of copies of $\mathcal{L}_r(b_r)$ and equals that language. Constant initialization uses b_r , giving equality in RS_E . $\square$

Lemma S3.4 (E': update-stage fluents suffice). *Suppose an UPD monitor references only declared fluents whose initiating and terminating labels are update commands: `stopOldSpec`, `startNewSpec`, `reconfigure` (including their indexed forms), or `hotSwapIn`. No such label occurs before actual entry. If the constant initializer is the non-error boundary state b_r obtained from compiled state zero by consuming `hotSwapIn` once, then it satisfies RS_E with equality at every actual entry.*

Proof. Before entry, no defining update label occurs, so all referenced fluents retain their declared initial values. The fresh E monitor therefore starts at state zero and consumes `hotSwapIn` once, reaching b_r . Every actual entry has reference language $\mathcal{L}_r(b_r)$, so the reference intersection equals that language. $\square$

The E' check reads the expanded initiating/terminating sets of each referenced fluent from the saved model exports, checks the constant initializer and its non-error boundary, and certifies 89/135 UPD occurrences. It uses the actual game's exclusion of pre-entry update commands; the older free-history diagnostic admits such commands and is retained unchanged. A name such as `stopPump` is an ordinary plant event, not an update command.

The historical E' condition leaves 46 R1 occurrences unverified because they reference ordinary-action predicates. Attaching different observer vectors to a fixed DFA does not establish their reference languages. The additional source-specific check below proves their entry interpretation without changing those historical records.

Lemma S3.5 (E: guarded-invariant entry initialization). *For a fresh entry-scoped requirement $\square(G \Rightarrow \neg A)$, suppose G is initially false, changes only on actual update commands excluded from old-endpoint histories, and is unchanged by entry. Let A be a disjunction of ordinary-event predicates, each reset by*

entry. If the saved DFA has the two-state invariant transitions described below and its constant initializer is the non-error false-guard state obtained by consuming entry once from its default initial state, then the initializer satisfies RS_E with equality at every actual entry.

Proof. No update command occurs before entry, so G is false there. Every possible value of the action predicates satisfies the initial implication. Entry leaves G false and resets all action predicates; the fresh pure invariant has no pending obligation from an earlier position. The resulting continuation language is thus identical for every entry valuation and equals that of the supplied boundary state. The compatible entry set is nonempty at an actual modeled entry, so intersecting these equal languages establishes exactness. $\square$

The added checker reads the original source formulas, expands action aliases, and checks their correspondence to saved monitor/observer exports. For each R1 occurrence, it requires one default-false declared guard, nonempty disjoint initiating/terminating sets consisting only of actual update commands, and ordinary-event predicates reset by entry. It checks source/export equality of these sets, a constant non-error boundary initializer, and all DFA transitions against the two-state invariant: initiating commands set the guard, terminating commands clear it, a forbidden ordinary event when the guard holds enters ERROR, and other events stutter. ERROR absorbs all labels; absent local-alphabet edges become ERROR and labels outside that alphabet stutter, matching the adapter. All 46 monitors omit the entry label from their local alphabets, so consuming entry stutters at the false-guard state.

Inspection of namespace validation and endpoint construction, together with the saved successful input-validation results, establishes that old histories contain only ordinary events. The adapter computes the boundary value once and installs it directly at every old root; initialization does not consume entry a second time. Seven negative mutations test rejection of a wrong guard value, non-update guard event, guard-changing entry, missing declared edge/label, non-rejecting forbidden event and missing entry reset. The checker verifies all 46 occurrences across 4,722 state-label pairs and 854 possible action-predicate entry valuations. These are static validation counts, not extra synthesis trials or enumerated physical histories. S4 preserves the older free-history and E' results, and the new CSV labels its distinct sufficient condition. The new proof uses the existing E meaning; it does not establish the history-inclusive H interpretation or application-level conformance.

S3.2 Lifecycle induction and residual meanings

At entry, each component is old, all update operations are pending, old monitor states are inherited from the reachable monitored endpoint, new monitors are inactive, and update-interval monitors receive the supplied ι_r . This establishes the version/pending equivalence and $t \in \text{ActiveM}(P) \iff \mu(t) \neq \perp$.

An ordinary step changes the active product and advances exactly its active monitors, preserving tags and pending operations. A transfer changes one old tag to new, selects every supplied transfer outcome, consumes its pending operation, and advances all active monitors once. A stop removes its old monitor and pending operation while advancing the other active monitors; the stopped monitor does not consume its own stop. A start advances the previously active monitors and installs its new monitor from the residual after its activation boundary, so the new monitor does not consume that boundary twice. The pending set enforces one-shot execution. The non-preemption guard only makes an update bucket empty; it does not alter any successful transition. Induction therefore preserves both invariants and the declared observation intervals.

Old monitors inherit their history from a verified old endpoint. For an activated requirement r , let $h \in H_r(\xi_{act})$ be its supplied activation history and let w be a subsequently observed monitored suffix. Policy safety ensures $w \in \mathcal{L}_r(\iota_r(\xi_{act}))$. The external RS premise gives

$$\mathcal{L}_r(\iota_r(\xi_{act})) \subseteq \bigcap_{h \in H_r(\xi_{act})} \{w : hw \in \text{SafePref}_r\}.$$

Thus the combined observed history satisfies the supplied requirement meaning. For A, this step quantifies over the admissible safe histories and establishes the chosen post-activation obligation; it does

not certify a violating pre-activation trace under H. For E, it uses the fresh entry-scoped reference languages and requires the independently justified entry initializer. Conservative residuals may remove legal continuations and can make a supplied game lose; algorithmic completeness is relative to that supplied game. Type, domain, and non-error checks on ι_r do not establish RS or the accuracy of SafePref_r as a description of intended application requirements.

At goal, every update operation has occurred, all new monitors are active, old monitors are inactive, and update monitors are safe. Equality of component and new-monitor projections with $\kappa(q)$ carries the new monitored continuation into the verified endpoint. The entry/load boundary contributes no ordinary observation, and update-interval monitoring ends at goal. This establishes the model-level interval safety used by the synthesis theorem.

S3.3 Execution correspondence

Assume the main paper's A1–A4 and a phase-disjoint observation map α . A4 gives an atomic entry from an actually reachable old state z to $\text{Init}(z)$; all-root coverage places it in the certified domain. Induct on observed update-game events. At a non-goal, A2 disables every unselected controllable, preserves all possible uncontrollables, and reports every remaining outcome within the policy-permitted complete bucket. For commands, A3 supplies one finite, failure-free, atomic event and its modeled successor identity at a quiescent source. Hence the next observation maps to a complete Post successor allowed by the policy. The certificate supplies closure, safety and a strict rank decrease.

A2 also supplies eventual observation of a next event while at a non-goal. A rank initially r permits at most r such decreases. No positive-rank deadlock is compatible with A2, so the execution reaches a certified goal. Goal recognition establishes quiescence from physical enabledness, rather than from a partial search table. Ordinary controllables are disabled there; A4 performs the finite atomic load of $\kappa(q)$ and the control switch. A1 maps the subsequent endpoint observations to the verified new closed loop. The lifecycle induction and externally supplied RS give safety for each declared interval relative to its specified residual meaning.

This is a conditional lifting theorem for an observed execution. A2 supplies concrete progress, A3 excludes internal τ , unobservable update outcomes and divergence within an update, and A4 supplies the atomic boundaries. The returned artifact is a finite policy/rank/handover table interpreted by an execution platform satisfying these premises. The linked LTS checker verifies the abstract boundary composition; the Java entry operation is named `hotSwapIn`, and its completion-side atomic quotient is `hotSwapOut`. The paper uses the neutral boundary names `entry` and `load`, respectively. The naming correspondence changes neither the game nor the implementation.

S4 Supplementary validation and cost tables

Historical RS coverage. The following generated table preserves the earlier H/A/E' checks. Its 46 unverified R1 entries describe that older sufficient condition; they are not the classification of the additional guarded-invariant check in Lemma S3.5. **RS sufficient-condition coverage**

Counts are requirement occurrences over base/R1/R2; repeated new requirements remain in the denominator. The independent free product includes absorbing ERROR and evolving fluent values after error. A failed sufficient check leaves RS untested and does not establish a reachable contract violation.

Model	New / upd	H checked	H FD	H exact	H untested	A sound ^a	E exact ^b	A/E unverified
GSM	3 / 3	6	0	0	6	3	2	1
Industry	9 / 9	18	0	0	18	9	6	3
MetaSocket	3 / 3	6	0	0	6	3	2	1
PowerPlant	9 / 10	19	0	0	19	9	7	3
PC Arms=1	18 / 18	36	0	0	36	18	12	6
PC Arms=2	36 / 36	72	0	0	72	36	24	12
Railcab	21 / 18	39	0	0	39	21	11	7
Surveillance	21 / 21	42	0	0	42	21	14	7
Workflow	18 / 17	35	0	0	35	18	11	6

FD means that equal reachable fluent valuations determine the same monitor state, including ERROR. Exact additionally requires compatibility with the actual initializer: NEW uses its non-error lookup; UPD uses a constant monitor state after `hotSwapIn`. Per-requirement state counts, two conflicting histories, and update-boundary checks are in `rs-requirements.csv`. ^aAll 138 NEW occurrences satisfy RS under A: the checked safe-prefix language condition gives equality when the actual safe-history set is nonempty; with an empty set, inclusion is automatic. Plant-level nonemptiness is not established, so exactness remains conditional. ^bThe current E column uses the syntactic sufficient condition (Lemma E'): every referenced fluent is declared and affected only by update labels, which cannot occur before entry; the constant initializer must match the non-error state after one `hotSwapIn`. Remaining event/plant predicates retain unverified reference-language comparisons. Earlier E diagnostic columns and all H/A results are unchanged. Update labels occur at most once in the new prefix explorations; continuation-language equivalence is checked over all words, a stronger condition.

Direct frontend lookup validation. We materialized the normal frontend's NEW lookup objects for all 27 application inputs using the same frozen JAR and byte-identical inputs. Each separate JVM ran once with a 2 GiB heap and 120 s limit; all 27 completed, with no retry or omitted target. Frontend compilation constructs the endpoint controllers, but the exporter neither runs FG synthesis nor expands physical closure or the activation domain. These are functional checks, not performance samples or recovered historical JVM objects.

An independent Python checker matched all 138 tables and their 1,587 entries (900 non-error, 687 ERROR). It verified actual observer order, alphabets, 408 observer columns and 31,176 transitions against the saved definitions. It reconstructs the full monitor/observer table over repeatable update events, includes the first ERROR frontier, applies ERROR priority and compares every key and value. A second reconstruction uses the earlier check's alphabet and order: all 138 alphabet differences and 114 column permutations are reconciled explicitly. The resulting non-error values and ERROR counts match the earlier CSV.

Five mutations of copies of real exports are detected: a safe value changed to ERROR, the converse, a deleted key with adjusted count, an omitted observer transition, and permuted columns without matching key permutation. Two separate Python unit cases cover initially-true normalization and ERROR priority. Actual data contain no initially-true columns, safe/ERROR key collisions or multiple safe states needing nontrivial language-equivalence checks, so these Java branches are not validated by the 27 exports. Physical-state decoding, full activation-domain construction and actual safe-history nonemptiness remain unverified. The complete inputs, commands, tables, comparison and negative checks are in `evidence/frontend_lookup_demo/`; these files await public release.

Additional entry-invariant evidence. The separate static checker `verify_r1_entry_invariant.py` verifies the guarded-invariant condition for R1 in GSM (1 occurrence), Industry (3), MetaSocket (1), PowerPlant (3), PC Arms=1 (6), PC Arms=2 (12), Railcab (7), Surveillance (7), and Workflow (6): 46/46 in total. Its CSV records each source formula and guard line, saved export selector, original run’s input validation and update labels, and the old E' flags without altering them. The seven negative mutations are separate checker tests. No synthesis decision, input formula, measurement or historical diagnostic was replaced. These new check files await integration into the public artifact release.

Benchmark contract coverage

Each triple is base/R1/R2. Transfer fan-out and loadable endpoint counts come from preserved first-trial outputs; component/requirement counts and explicit precedence are cross-checked against the parsed definitions.

Model	Components	Multi-target	Initializers	Prec.	$ Z_{\text{load}} $
GSM	1	No	1/2/3	0	9
Industry	3	No	3/6/9	0	81
MetaSocket	2	No	1/2/3	0	4
PowerPlant	2	No	3/6/10	0	4
PC Arms=1	1	No	6/12/18	0	9
PC Arms=2	2	No/No/-	12/24/36	0	137
Railcab	4	Yes	7/14/18	0	71
Surveillance	2	No	7/14/21	0	107
Workflow	5	No	6/12/17	0	21

Multi-target reports whether a transfer source has more than one outcome. Initializers count NEW plus UPD requirements; NEW table lookup and UPD constant boundary initialization are distinguished in the RS table. PC Arms=2/R2 multi-target status is unmeasured (-): preserved runs do not contain the final transfer census. Zero explicit precedence does not mean transition monitors impose no ordering constraints.

Censored JVM attempts and elapsed time

The 1,200 s cap covers the whole JVM invocation, including startup, frontend and endpoints, solving, checks and output. For each Direct-Full timeout with five consistent FG completions, $b = 1200 / \text{median}(t_{\text{FG,elapsed}})$ is a lower bound on that censored attempt relative to the FG elapsed median, not a Direct-Full median or a solver-only speedup. The last column records the $1200 / \text{median}(t_{\text{FG,solver}})$ quotient for transparency; it has no solver-time lower-bound interpretation because the cap includes non-solver work. Displayed bounds are rounded downward.

Model	Target	FG elapsed (s)	FG solver (s)	$b (\geq)$	Cap/solver
Industry	r1	563.93	562.13	2.12	2.13
PC Arms=1	r1	1.32	0.08	910.06	15125.54
PC Arms=2	base	19.36	2.61	61.99	459.89
PC Arms=2	r1	19.86	2.60	60.43	461.22
Railcab	base	2.23	0.17	539.14	7090.31
Railcab	r1	5.25	3.18	228.62	377.56
Surveillance	base	4.34	2.01	276.32	596.89
Surveillance	r1	21.16	18.64	56.71	64.37
Surveillance	r2	39.89	37.07	30.08	32.38
Workflow	base	7.06	0.11	169.92	10810.32
Workflow	r1	7.17	0.33	167.45	3688.87
Workflow	r2	23.05	16.06	52.05	74.70

Workflow/R2 uses the authorized supplementary attempt; the original operational interruption remains in the provenance table. Industry/R1 compares an unresolved attempt with a completed LOSS decision. The table measures time to decide the supplied game, not time to return a controller.

Total JVM elapsed seconds: five-run median [min, max]

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	W 0.913 [0.913, 0.913]	W 0.913 [0.913, 0.914]	W 0.913 [0.912, 0.915]	W 0.913 [0.913, 0.916]	W 1.22 [1.11, 1.22]
	R1	W 0.913 [0.912, 0.914]	W 0.913 [0.913, 0.914]	W 0.913 [0.913, 0.914]	W 0.913 [0.913, 1.01]	W 1.22 [1.22, 1.22]
	R2	W 0.914 [0.912, 0.915]	W 0.914 [0.914, 0.914]	W 0.913 [0.913, 0.914]	W 0.914 [0.913, 0.914]	W 1.12 [1.11, 1.22]
Industry	BASE	W 1.92 [1.92, 1.92]	W 16.3 [16, 16.5]	W 2.22 [2.22, 2.33]	W 156 [154, 168]	N/M
	R1	L 564 [536, 568]	TO	L 462 [368, 514]	TO	N/M
	R2	W 2.93 [2.93, 2.93]	W 4.75 [4.74, 4.95]	W 2.23 [2.22, 2.33]	W 29.8 [29.5, 30.1]	N/M
MetaSocket	BASE	W 0.912 [0.813, 0.914]	W 0.914 [0.91, 0.915]	W 0.914 [0.911, 0.914]	W 0.913 [0.91, 0.914]	W 1.22 [1.21, 1.32]
	R1	W 0.913 [0.913, 0.913]	W 0.914 [0.914, 0.915]	W 0.914 [0.912, 0.914]	W 0.913 [0.912, 0.913]	W 1.42 [1.42, 1.43]
	R2	W 0.913 [0.913, 0.914]	W 0.913 [0.913, 0.914]	W 0.914 [0.912, 0.915]	W 0.914 [0.913, 0.915]	W 1.22 [1.22, 1.22]
PowerPlant	BASE	W 1.01 [1.01, 1.02]	W 1.62 [1.52, 1.62]	W 1.01 [1.01, 1.02]	W 2.83 [2.83, 2.83]	W 19.2 [18.3, 27]
	R1	W 1.12 [1.12, 1.12]	W 2.13 [2.02, 2.13]	W 1.22 [1.12, 1.22]	W 6.56 [6.45, 6.66]	W 898 [807, 912]
	R2	W 1.22 [1.22, 1.22]	W 1.32 [1.32, 1.32]	W 1.22 [1.21, 1.22]	W 2.12 [2.02, 2.13]	W 9.29 [8.88, 9.38]
PC Arms=1	BASE	W 1.22 [1.21, 1.22]	W 27.6 [27.2, 28.5]	W 2.03 [2.02, 2.12]	W 159 [157, 160]	N/M
	R1	W 1.32 [1.31, 1.32]	W 191 [188, 193]	W 4.43 [4.25, 4.55]	TO	OOM
	R2	W 7.56 [7.46, 8.27]	W 13.6 [13.6, 14.1]	W 2.43 [2.42, 2.53]	W 79.4 [78.5, 79.6]	N/M
PC Arms=2	BASE	W 19.4 [19.2, 19.9]	TO	TO	TO	OOM
	R1	W 19.9 [19.7, 20.3]	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	W 2.23 [2.22, 2.23]	TO	W 4.24 [4.24, 4.25]	TO	OOM
	R1	W 5.25 [5.24, 5.25]	TO	W 8.88 [8.76, 9.07]	TO	OOM
	R2	W 17.9 [17.8, 17.9]	W 41.2 [41.2, 41.4]	W 6.36 [6.35, 6.46]	W 285 [282, 287]	OOM
Surveillance	BASE	W 4.34 [4.34, 4.44]	TO	W 5.15 [4.85, 5.25]	TO	OOM
	R1	W 21.2 [20.8, 21.3]	TO	W 129 [129, 134]	TO	OOM
	R2	W 39.9 [39.9, 40.4]	TO	W 18.8 [18, 19.1]	TO	OOM
Workflow	BASE	W 7.06 [6.96, 7.06]	W 518 [501, 528]	W 7.06 [7.06, 7.16]	TO	OOM
	R1	W 7.17 [7.06, 7.26]	TO	W 7.06 [6.96, 7.17]	TO	OOM
	R2	W 23.1 [22.8, 23.4]	W 30.9 [30.4, 31.6]	W 10.2 [10.1, 10.3]	TO [‡]	OOM

Incomplete cells retain their original display statuses and receive no timing estimate. Elapsed time is measured by the harness monotonic clock; timeout termination overhead is excluded from the conservative 1,200 s numerator above.

RQ1’s 92 jobs are 41 semantic models under two methods and five endpoint-contract fixtures under two methods. Four invalid fixtures reject each method; the remaining contract fixture wins. Related variants share source models, so job count is not a count of independent applications. RQ2 has 14 jobs over eight finite LTS files. The main paper’s capability table includes two permanent branch-action bans and each neutral control. All failures and invalid outcomes are retained. The three quiescence probes are separate from these fixed denominators. Historical PRISM results use the previous goal rule and are not included in the confirmed counts.

Ten original semantic models have 31 variants that rename or reorder elements, add unreachable states, or permute components. The code passes 41 focused tests, including five boundary tests; the broader run passes 99 tests with one opt-in integration test unexecuted.

The generated tables report all RQ3 model/method cells and metrics. Timing summaries require five valid, decision-consistent completions; other groups retain their statuses. Mac pilot and Industry/R1 diagnostic measurements are excluded from Xeon timing comparisons. The controlled-family

Table S5: Full main-campaign outcome grid for all 135 instance–method cells.

Model	FG lazy	Direct-Full	Eager	Update-first	Legacy
GSM	W/W/W	W/W/W	W/W/W	W/W/W	W/W/W
Industry	W/L/W	W/T/W	W/T/W	W/L/W	N/N/N
MetaSocket	W/W/W	W/W/W	W/W/W	W/W/W	W/W/W
PowerPlant	W/W/W	W/W/W	W/W/W	W/W/W	W/W/W
PC Arms=1	W/W/W	W/T/W	W/W/W	W/W/W	N/O/N
PC Arms=2	W/W/T	T/T/T	T/T/T	T/T/T	O/O/O
Railcab	W/W/W	T/T/W	T/T/W	W/W/W	O/O/O
Surveillance	W/W/W	T/T/T	T/T/T	W/W/W	O/O/O
Workflow	W/W/W	T/T/T [‡]	W/T/W	W/W/W	O/O/O
Decided / 27	26	14	17	24	9
WIN / LOSS	25/1	14/0	17/0	23/1	9/0
TO / OOM / N	1/0/0	13/0/0	10/0/0	3/0/0	0/13/5

Each triple is Base/R1/R2; all 135 cells are retained. W/L: WIN/LOSS; T: timeout; O: out of memory; N: not measured before synthesis (author-side capture cap). [‡]: same-setting supplementary timeout, with the interrupted original preserved. PC Arms=2/R2 also times out in a separate 3,600 s FG trial. Legacy uses its native objective.

Table S6: Confirmed-semantic checks. Denominators exclude inapplicable checks, not failed cases.

Check	RQ1	RQ2
Planned jobs	92	14
WIN / LOSS / invalid	36 / 48 / 8	10 / 4 / 0
Agreement with raw-LTS expectation	92/92	14/14
Internal certificate, eligible jobs	84/84	14/14
Separate exhaustive checker, eligible jobs	84/84	14/14
Atomic Link check, WIN jobs	36/36	10/10

table reports first-trial structure only. The five Legacy N/M (capture cap) cells are interrupted before synthesis by the author-side M9Mt sSnapshot instrumentation’s fixed 250,000-state cap. They give no evidence of Legacy capability or solver failure. Saved process statuses remain unchanged; N/M is an evidence-based display annotation.

Industry/R1 diagnostic. One Mac run of the frozen JAR enables the separate exhaustive checker and confirms UNREALIZABLE by its reachable-state fixed point (110,992 states, 873,132 successor queries, 211,940 outcomes). Its exported game stops at goals, giving 110,825 states; a Python fixed point on this export identifies 89 winning and 42 losing entries among 131 roots and checks every losing-region closure condition. These counts concern correctness, not performance.

The stored graph identifies a losing entry $Q77863$: TOR has already completed once, DSD1 has not, and pending inputs have been received. All nine update events can execute in stop–reconfigure–start order, reaching safe, uncontrollable-quiescent $Q19580$ with no pending update event, but with a signature outside the new endpoint’s compatible set. From there the environment can sustain

$$Q19580 \xrightarrow{\text{validatedSD1}} Q29028 \xrightarrow{\text{dsd1ok.0}} Q21628 \xrightarrow{\text{reviewSD1}} Q19204 \xrightarrow{\text{receivedSD1}} Q19580.$$

Validation and review are controllable; rejection and receipt are uncontrollable. At each safe state, alternative GF1/TOR validations violate an active monitor; declining validation at quiescent $Q19580$ deadlocks. The four safe states and eight unsafe alternatives form a 12-state spoiling certificate. R1 disallows ordinary controllable events between stopping the old requirements and starting the new ones. The global losing-region check, rather than this illustrative update order alone, rules out alternative policies. The exported game, command, outputs, collector diagnostics, and reproducible graph analysis are in `raw/industry-r1-independent/`. Update-command preemption alone cannot

remove a cycle with no pending update commands; whether a different command/handover contract avoids this region remains unverified.

Monolithic cross-check: original-source lab build versus this fork. The input is the monolithic model from which the fine-grained adaptation was derived. Its bytes match the original-source lab build’s bundled model after the stated transition selection; identity with a final published benchmark release remains unconfirmed. Only the two transition-selection lines change:

```
-// transition = T_Empty,
-   transition = T,
+   transition = T_Empty,
+   //transition = T,
```

Whitespace is normalized in this display. The definitions remain

$$T = (StopOldSpec \wedge TORDone) \Rightarrow (\neg cancelGF1 \wedge \neg approveGF1 \wedge \neg startNewSpec),$$

$$T_{Empty} = (StopOldSpec \wedge \neg StartNewSpec) \Rightarrow \neg AnyAction.$$

The supplied logs both show a 64-state old controller; the fork also reports a 240-state mapping product. The original-source lab-build log reports a 2,312-state update controller, whereas the fork reports no controller. The following selected lines retain formula names, construction sizes and decisions; gaps omit other formulas and resource measurements. The lab-build log has no `Mode:` line, so none is added. The executable is a lab-maintained build of the original DUCS source, not an identified official TSE release; these runs are decision evidence, separate from the main RQ3 table.

Original-source lab-build log:

```
Processing formula for update: P_DO_NOT_SEND_TWICE_UPD_OLD
Processing formula for update: P_NEW_TOR_POLICY_UPD_NEW
Processing formula for update: P_NEW_DSD1_POLICY_UPD_NEW
Processing formula for update: P_NEW_DO_NOT_SEND_TWICE_UPD_NEW
Processing formula for update: T_Empty
Environment states:14763
Solving safety goals for the controller synthesis
Environment states after safety: 3134
Synthesising GR
Environment after safety is deterministic
Solving a deterministic controller synthesis
Controller [2312] generated successfully.
```

Fork log:

```
Mode: Traditional DUC (legacy update events)
Processing formula for update: P_DO_NOT_SEND_TWICE_UPD_OLD
Processing formula for update: P_NEW_TOR_POLICY_UPD_NEW
Processing formula for update: P_NEW_DSD1_POLICY_UPD_NEW
Processing formula for update: P_NEW_DO_NOT_SEND_TWICE_UPD_NEW
Processing formula for update: T_Empty
Environment states:33093
Solving safety goals for the controller synthesis
Environment states after safety: 15177
Synthesising GR
Environment after safety is deterministic
Solving a deterministic controller synthesis
There is no controller for model UpdCont for the given setting.
```

The full logs and both unchanged input files are in `experiments/industry_r1_legacy_monolithic_check/`. These results rule out interpreting the FG LOSS as general unrealizability of R1. The FG requirement is universal completion from all 131 entries into a compatible verified endpoint at quiescence. Published DUCS instead synthesizes a controller that continues enforcing the new specification after the switch; it takes no fixed new-controller endpoint as input.

Source explanation of the cross-check. We compare the fork’s synthesis, environment construction and GR-goal code with the official MTSA source at commit d10ba71f. The inspected official revision is separate from the supplied lab build (source commit e1bf040); the latter JAR is the one used for the recorded GUI result. Both constructions retain the old closed loop and allow update entry throughout its reachable states; a GR solver’s single initial-state check covers that entire pre-update behavior. Both traditional paths synthesize continued control under old/transition/new safety and a GR progress objective. A persistent only-turn-on fluent for the entry event is the GR assumption; persistent stop/start/reconfigure fluents give the three guarantees. Thus the input keyword `nonblocking` does not make this an existential-only completion check. Neither traditional path checks a fixed new-controller connection or quiescent handover. Its separate OTF branch uses `DirectedController\allowbreakSynthesisDUC` and an endpoint connection map; neither is used here.

The upstream formula builder uses the model’s `StopOldSpec/StartNewSpec`. The fork substitutes internal progress fluents (prefix `__DUC__`) when reconstructing the violation formulas. Its seven old-safety fluents are `TORDone`, `DSD1Done`, `Finished`, the three validation-event fluents, and internal `StopOldSpec`. The seven new-safety fluents replace `TORDone` by `NewTORDone` and internal `StopOldSpec` by internal `StartNewSpec`: their union has nine. `T_Empty` adds the model’s `StopOldSpec/StartNewSpec` and eight controllable-event fluents, sharing three validation fluents. Hence the logged total is $9 + 10 - 3 = 16$; these are not 16 additional endpoint obligations.

The fork’s update-entry constant is `hotSwapIn`. The model’s two progress fluents still reset on `beginUpdate`. `removeTopStates` composes fluent automata even when their labels lie outside the base environment alphabet. Consequently `beginUpdate` becomes an independent uncontrollable event, resetting the user fluents without resetting the internal ones. With the upstream entry label `beginUpdate`, that reset instead synchronizes with entry. A finite Python reconstruction of this specific input, without modifying or executing either Java tool, gives:

Product construction	States	Transitions
Old controller	64	148
Mapping environment	240	1,108
Fork entry/reset mismatch	33,093	257,314
Synchronized entry/reset	14,763	99,676
Fork after safety and at-most-once monitors	15,177	90,250
Synchronized after safety and at-most-once monitors	3,134	11,232

Both meta-product state/transition pairs exactly match the respective saved logs, as do the 15,177 versus 3,134 final safety-state counts. The synchronized final transition count is reconstructed, not printed by the lab-build log. The mapping count is reconstructed from the supplied model and printed by the fork; the lab-build log does not print that intermediate count. The reconstruction and its assertions are provided as `check_legacy_product.py` beside the two inputs and logs. It checks a concrete encoding discrepancy, not a new result for the frozen 27-instance campaign. The mismatch also explains the fork’s negative decision: immediately after `hotSwapIn`, the safe `beginUpdate` self-loop can repeat forever without consuming any stop/start/reconfigure event. The GR entry assumption stays true while all three progress guarantees stay false. This uncontrollable trace survives safety and at-most-once filtering. It is a label-compatibility discrepancy, not a new-controller handover requirement. No input, configuration, JAR or synthesis source was changed to obtain this explanation.

The source model’s `AnyAction` includes eight controllable events; its filename and `T_Empty` label alone do not establish identity with the published benchmark version.

Aligned-entry execution and reference-build identity. The reference is a lab-maintained build of the original DUCS source at commit e1bf040b83e36b36308bce22e280133682986250, using the same 672,505,060-byte JAR as the recorded GUI execution. Its SHA-256 is

4ab198ba14c9f0f829c36808025aefc063
e2a3a0836679490179cc407e6133e5

The eight monolithic model files agree byte-for-byte between that commit’s resources, compiled resources, and the JAR. This identifies the lab build and its inputs without asserting an official published release identity.

A separate fork input changes only the three fluent reset lines for StopOldSpec, StartNewSpec and Reconfigure, replacing `beginUpdate` by `hotSwapIn`. One Mac execution of the frozen fork JAR through its Traditional path returns a 2,312-state controller, agreeing with the reference result; the meta-product and final safety input also match at 14,763 and 3,134 states. The earlier mismatched-input failure remains preserved. This is a decision-only fidelity check, excluded from RQ3 and its performance measurements. Input diff, command and original outputs are retained under `experiments/legacy_fidelity/`; the returned monolithic reference campaign is reported separately below.

The separate adapter pilot checks Industry under two monolithic conditions (states/transitions below); all four cells return WIN. The aligned fork’s T_{Empty} cell reuses that one execution.

Input	Original-source lab build	Aligned fork
Supplied T	3,050 / 9,496	3,050 / 9,496
T_{Empty}	2,312 / 6,615	2,312 / 6,615

Two initial adapter failures are preserved: synthesis produced a controller, but an incorrect Java type check rejected the outer composite. Correcting only that adapter yielded the two successful reference-tool cells. The final adapter additionally classifies an explicit no-controller result wrapped in a composition exception; four classification assertions check that branch. These Mac checks supply no performance evidence. The separate Xeon campaign has 21 conditions per tool (15 supplied targets and six named empty-transition alternatives); its returned results remain outside the main RQ3 table.

Returned legacy-fidelity reference experiment. Both Windows campaigns have completed: the lab-maintained original-source build and this fork use mechanically generated monolithic conditions, with the three entry-fluent resets aligned. They remain reference tools with different contracts from FG; none of these conditions is a new RQ3 cell. Table S7 reads their saved summaries and per-trial logs. The collector retains raw process statuses; its local CSV is byte-identical to the output of the deployed collector on the returned inputs. The separately produced Xeon comparison CSV and final run-all transcript are also retained; all six driver stages complete successfully, while the recorded fork exceptions remain failures. The locally regenerated and Xeon-produced CSVs are byte-identical.

Table S7: Separate monolithic reference experiment: all 21 conditions. S denotes SUCCESS (controller returned); sizes are first-trial states/transitions. Each completed cell has five consistent SUCCESS decisions; sizes may vary across repetitions (below). N/M labels annotate raw CRASH records, without changing them.

Input / condition	Original-source lab build	Fork legacy path	Comparison
GSM supplied	S 86/119	N/M (fork parser)	N/M
GSM empty	S 92/117	N/M (fork parser)	N/M
Industry supplied	S 3,050/9,496	S 3,050/9,496	MATCH
Industry empty	S 2,312/6,615	S 2,312/6,615	MATCH
MetaSocket	S 36/59	S 36/59	MATCH
UPD64_128			
MetaSocket	S 36/59	S 36/59	MATCH
UPD64_64COM			

Input / condition	Original-source build	lab Fork legacy path	Comparison
MetaSocket UPD128_64	S 36/59	S 36/59	MATCH
MetaSocket UPD128_128COM	S 36/59	S 36/59	MATCH
MetaSocket UPD64COM_64	S 36/59	S 36/59	MATCH
MetaSocket UPD64COM_128COM	S 36/59	S 36/59	MATCH
MetaSocket UPD128COM_128	S 36/59	S 36/59	MATCH
MetaSocket UPD128COM_64COM	S 36/59	S 36/59	MATCH
PowerPlant supplied	S 79/141	N/M (fork parser)	N/M
ProductionCell supplied	S 168/267	S 168/267	MATCH
ProductionCell empty	S 107/158	S 107/158	MATCH
Railcab supplied	S 448/803	S 443/796	SIZE- MISMATCH
Railcab empty	S 471/708	S 471/708	MATCH
Surveillance supplied	S 1,394/2,959	N/M (instrumentation)	N/M
Surveillance empty	S 665/1,414	N/M (instrumentation)	N/M
Workflow supplied	S 679/1,144	S 679/1,144	MATCH
Workflow empty	S 193/287	S 193/287	MATCH

The two campaigns retain 210 planned slots: 190 executed trials and 20 explicit skips following the five fork exceptions. Those exceptions occur before a legacy GR decision and give no evidence of legacy synthesis capability or failure.

Saved exception text: fork parser. `ltsa.lts.LTSEException:`

name already defined LeavesPumpOn

`ltsa.lts.LTSEException:`

name already defined T_NO_UPDATE_WHILE_SEND

Saved exception text: instrumentation. `java.lang.IllegalArgumentException:`

Derived old-action authority is inconsistent: weather.old

Tool	Condition	Rep. 1	Rep. 2	Rep. 3	Rep. 4	Rep. 5
Fork	ProductionCell / empty	107/158	102/150	107/158	102/150	107/158
Lab build	Railcab / supplied	448/803	448/809	449/805	443/796	442/797
Fork	Railcab / supplied	443/796	442/794	449/805	451/811	449/805
Lab build	Railcab / empty	471/708	471/708	471/708	429/651	471/708

These are all 4 tool/condition cells whose returned controller sizes vary across the five successful trials; other completed cells retain their first-trial sizes. The fifteen size matches and the five-state Railcab difference in the main text compare first trials only. In particular, the Railcab supplied lab-build trial 4

returns the same size as fork trial 1; a persistent tool-specific size difference or controller equivalence is not established.

Causes of unmeasured fork conditions. GSM reuses `T_NO_UPDATE_WHILE_SEND` for a property and an assertion (input lines 105/130); PowerPlant similarly reuses `LeavesPumpOn` (91/114). In `LTSCompiler`, the original parser’s uniqueness check consults assertion definitions through `getDefinition` (1889–1897). The fork additionally checks constraints through `getConstraint` (2054), rejecting these original inputs during parsing. This accounts for the three N/M (fork parser) entries, not a synthesis LOSS.

Surveillance declares `weather` only in `NewControllableActions`, not in its old/new alphabets or environment transitions. The fork derives `weather.old` from the declaration and inserts that derived label into the alphabet (`UpdatingControllerSafetySynthesizer`, 565–582, 612–615). Author-side `M9TraditionalSafetySemanticsSnapshot` requires both the derived label and its base action in the alphabet (519–525); the absent base `weather` triggers the saved exception. This capture follows construction of the old controller and meta-environment, but precedes update safety pruning and the legacy GR decision (safety synthesizer, 129–176). The two N/M (instrumentation) entries therefore provide no evidence of legacy synthesis capability or failure. No model, tool or JAR was changed and no trial was repeated to repair these outcomes.

Railcab size variation and unresolved cause. For supplied Railcab, both logs report 1,763 states before safety pruning and 1,582 afterward, then use nondeterministic controller synthesis. The first-trial difference is already present when the strategy is converted to an MTS and unreachable states are removed, before final `CompactState` conversion or outer composition. The inspected subset construction, knowledge-game solver and strategy-to-MTS paths share their operative code; preprocessing, fluent-valuation handling and enumeration still differ. The saved evidence does not establish corresponding safety graphs, valuations or strategies. Moreover, controller sizes vary within each tool across repetitions, with some sizes shared across tools. The cause of that variation and the first-trial difference remains unresolved after bounded static inspection; neither a persistent fork effect nor controller equivalence is inferred.

Table S8: Reference measurements only. End-to-end JVM wall seconds and sampled process peak RSS (GiB): median [min,max] only for five complete, consistent runs. No cross-tool ratios are computed.

Input / condition	Lab wall (s)	Fork wall (s)	Lab RSS (GiB)	Fork RSS (GiB)
GSM supplied	0.61 [0.609, 0.61]	N/M	0.102 [0.0995, 0.102]	N/M
GSM empty	0.711 [0.71, 0.712]	N/M	0.101 [0.0997, 0.105]	N/M
Industry supplied	12.7 [12.6, 12.7]	16.7 [16.3, 16.8]	1.91 [1.6, 2.05]	2.02 [1.97, 2.11]
Industry empty	4.64 [4.44, 4.74]	7.66 [7.46, 7.77]	0.814 [0.738, 0.821]	1.81 [1.73, 1.89]
MetaSocket UPD64_128	0.611 [0.609, 0.612]	1.21 [1.11, 1.21]	0.0996 [0.0971, 0.103]	0.14 [0.137, 0.147]
MetaSocket UPD64_64COM	0.609 [0.608, 0.61]	1.21 [1.21, 1.22]	0.0996 [0.0995, 0.0998]	0.141 [0.139, 0.144]
MetaSocket UPD128_64	0.61 [0.609, 0.611]	1.21 [1.21, 1.21]	0.0995 [0.0981, 0.0998]	0.142 [0.14, 0.143]
MetaSocket UPD128_128COM	0.61 [0.609, 0.61]	1.21 [1.11, 1.22]	0.0997 [0.0971, 0.103]	0.141 [0.14, 0.142]
MetaSocket UPD64COM_64	0.61 [0.609, 0.611]	1.21 [1.11, 1.21]	0.1 [0.098, 0.103]	0.144 [0.137, 0.145]
MetaSocket UPD64COM_128COM	0.609 [0.609, 0.611]	1.21 [1.21, 1.22]	0.1 [0.0995, 0.103]	0.143 [0.139, 0.15]
MetaSocket UPD128COM_128	0.61 [0.608, 0.612]	1.21 [1.11, 1.21]	0.1 [0.0997, 0.103]	0.145 [0.141, 0.147]
MetaSocket UPD128COM_64COM	0.61 [0.608, 0.61]	1.21 [1.21, 1.22]	0.102 [0.1, 0.103]	0.144 [0.141, 0.144]

Input / condition	Lab wall (s)	Fork wall (s)	Lab RSS (GiB)	Fork RSS (GiB)
PowerPlant supplied	0.711 [0.71, 0.712]	N/M	0.105 [0.101, 0.109]	N/M
ProductionCell supplied	1.11 [1.11, 1.12]	1.92 [1.92, 1.92]	0.199 [0.191, 0.206]	0.303 [0.3, 0.308]
ProductionCell empty	1.12 [1.11, 1.12]	1.92 [1.82, 1.92]	0.196 [0.182, 0.204]	0.309 [0.297, 0.322]
Railcab supplied	1.62 [1.62, 1.72]	2.63 [2.62, 2.63]	0.482 [0.472, 0.496]	0.703 [0.696, 0.711]
Railcab empty	1.82 [1.82, 1.92]	3.03 [2.93, 3.03]	0.558 [0.537, 0.566]	0.682 [0.671, 0.689]
Surveillance supplied	6.35 [5.84, 6.45]	N/M	0.862 [0.852, 0.867]	N/M
Surveillance empty	10.7 [10.4, 10.8]	N/M	0.929 [0.877, 0.95]	N/M
Workflow supplied	9.37 [9.17, 9.48]	12.3 [12.1, 12.4]	1.03 [0.974, 1.1]	1.98 [1.96, 2.04]
Workflow empty	4.53 [4.43, 4.64]	5.65 [5.54, 5.94]	0.664 [0.631, 0.667]	1.8 [1.79, 1.94]

Internal times have different scopes: the lab-build adapter includes compilation, continued compilation and composition, whereas the fork exposes its solve-control-problem interval. The table instead uses end-to-end JVM wall time as reference information; these tools do not solve identical FG games and their times are not ratioed. The fork exceptions have one actual trial followed by four explicit skips; no median is reported. `experiments/legacy_fidelity/` holds the inputs, diffs, collector, renderer and raw evidence; the public package omits shaded JARs and the private delta’s `lib/`.

Separate validation-permitting contract variant. Keep the original Industry components, end-point controllers, transfer relations, roots, precedence and Post/goal rules. In each of the three R1 update-time properties, replace

$$\begin{aligned} & \Box(\text{AfterStopBeforeStart}_r \Rightarrow \neg \text{AnyAction}) \\ \text{by } & \Box(\text{AfterStopBeforeStart}_r \Rightarrow \neg(\text{AnyAction} \wedge \neg \text{validateGF1})). \end{aligned}$$

Thus the same one event, `validateGF1`, becomes permitted in those intervals; the other seven ordinary controllable labels stay forbidden. This assumes that gate-form validation is an acceptable operation during update. It changes the supplied contract, not the synthesis semantics, and its application-level acceptability is an external premise.

One Mac run with the frozen JAR returns WIN, passing the 214-state winning certificate, atomic Link and separate exhaustive checks (145,864 reachable states). A Python fixed point over the export confirms that the 131 roots match the original tuples and all win, including the original 42 losing entries. Its independently extracted rank certificate covers 273 states and 407 successor buckets; its different size reflects policy extraction, not a different decision.

At the illustrative losing root’s counterpart, a winning path reconfigures the three components, stops the three old requirements, starts the new no-duplicate requirement, and validates GF1. Each of the uncontrollable outcomes `adjustGF1`, `approveGF1` and `cancelGF1` resets the relevant TOR/DSD1 history fluents; receipt of GF1 and activation of the remaining two requirements then reach a compatible goal. All outcomes are retained in the certificate. The exact three-line LTS diff, executable checks, source-fluent definitions and raw evidence are in `experiments/industry_r1_contract_variant/`. This decision-only diagnosis supplies no timing comparison and is excluded from the 27-instance table; the original R1 LOSS remains unchanged.

Windows Xeon campaign; all 27 instances and five methods retained.
Discovered states (first trial)

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	31	75	32	75	–
	R1	34	114	34	115	–
	R2	34	68	46	68	–
Industry	BASE	272	1.1e+05	2470	2.8e+05	N/M
	R1	3.9e+04	TO	3.1e+04	TO	N/M
	R2	7717	2.8e+04	2875	5.1e+04	N/M
MetaSocket	BASE	8	71	11	82	–
	R1	8	88	11	95	–
	R2	16	31	17	34	–
PowerPlant	BASE	73	5634	101	8168	–
	R1	759	2.3e+04	911	2.4e+04	–
	R2	1330	2810	600	3594	–
PC Arms=1	BASE	83	4.1e+05	6018	4.1e+05	N/M
	R1	94	3.3e+06	2e+04	TO	OOM
	R2	5.6e+04	1.6e+05	9304	1.6e+05	N/M
PC Arms=2	BASE	2502	TO	TO	TO	OOM
	R1	2433	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	171	TO	1.4e+04	TO	OOM
	R1	1.3e+04	TO	3.7e+04	TO	OOM
	R2	9.8e+04	3e+05	3.8e+04	3.2e+05	OOM
Surveillance	BASE	5319	TO	1.4e+04	TO	OOM
	R1	9.6e+04	TO	6.7e+05	TO	OOM
	R2	1.8e+05	TO	1.2e+05	TO	OOM
Workflow	BASE	151	4.3e+06	173	TO	OOM
	R1	1098	TO	519	TO	OOM
	R2	1.5e+05	2.9e+05	3.3e+04	TO [‡]	OOM

Successor queries (first trial)

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	27	658	31	658	–
	R1	44	695	38	706	–
	R2	78	352	44	352	–
Industry	BASE	3336	1.4e+06	3.1e+04	5.1e+06	N/M
	R1	2.5e+05	TO	2.1e+05	TO	N/M
	R2	5.3e+04	2.1e+05	1.8e+04	7.3e+05	N/M
MetaSocket	BASE	12	190	18	289	–
	R1	12	201	18	257	–
	R2	24	53	22	79	–
PowerPlant	BASE	165	2.7e+04	242	4.8e+04	–
	R1	1927	6.4e+04	2619	6.6e+04	–
	R2	3118	7528	1149	1.4e+04	–
PC Arms=1	BASE	840	2.5e+06	5.2e+04	3.8e+06	N/M
	R1	852	1.2e+07	2e+05	TO	OOM
	R2	3.5e+05	8.5e+05	5.9e+04	1.4e+06	N/M
PC Arms=2	BASE	5e+04	TO	TO	TO	OOM
	R1	4.4e+04	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	1575	TO	1.4e+05	TO	OOM
	R1	9.8e+04	TO	3.7e+05	TO	OOM
	R2	4.4e+05	1.4e+06	1.6e+05	2.8e+06	OOM
Surveillance	BASE	3.8e+04	TO	1.1e+05	TO	OOM
	R1	4.8e+05	TO	5.5e+06	TO	OOM
	R2	8.8e+05	TO	5.8e+05	TO	OOM
Workflow	BASE	1131	3e+07	1329	TO	OOM
	R1	7142	TO	4499	TO	OOM
	R2	5.9e+05	1.1e+06	1.2e+05	TO [†]	OOM

Transition outcomes (first trial)

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	24	162	25	162	–
	R1	28	170	28	172	–
	R2	35	98	39	98	–
Industry	BASE	417	3.2e+05	5762	9.3e+05	N/M
	R1	7e+04	TO	5.7e+04	TO	N/M
	R2	1.2e+04	5e+04	4292	1.3e+05	N/M
MetaSocket	BASE	7	156	10	184	–
	R1	7	163	10	177	–
	R2	19	44	16	50	–
PowerPlant	BASE	105	2.1e+04	141	2.8e+04	–
	R1	1441	4.9e+04	1753	5e+04	–
	R2	2519	6024	873	7910	–
PC Arms=1	BASE	131	9.6e+05	1.6e+04	9.6e+05	N/M
	R1	134	5.7e+06	5e+04	TO	OOM
	R2	1.2e+05	3.4e+05	2e+04	3.4e+05	N/M
PC Arms=2	BASE	5615	TO	TO	TO	OOM
	R1	4987	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	216	TO	3.5e+04	TO	OOM
	R1	2.7e+04	TO	8.2e+04	TO	OOM
	R2	1.8e+05	5.7e+05	6.9e+04	6.2e+05	OOM
Surveillance	BASE	1.2e+04	TO	3.1e+04	TO	OOM
	R1	2.5e+05	TO	1.9e+06	TO	OOM
	R2	4.5e+05	TO	2.5e+05	TO	OOM
Workflow	BASE	149	1.2e+07	185	TO	OOM
	R1	2197	TO	999	TO	OOM
	R2	2.4e+05	4.6e+05	5.4e+04	TO [‡]	OOM

Solver time (seconds; five complete trials)

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	W 0.0224 [0.0208, 0.0236]	W 0.0332 [0.0323, 0.0362]	W 0.0233 [0.0222, 0.0235]	W 0.0416 [0.0398, 0.0421]	W 0.241 [0.234, 0.252]
	R1	W 0.0228 [0.0216, 0.0232]	W 0.0348 [0.0315, 0.0357]	W 0.0243 [0.0233, 0.0249]	W 0.045 [0.0438, 0.0472]	W 0.306 [0.303, 0.309]
	R2	W 0.0201 [0.0197, 0.0219]	W 0.0281 [0.0271, 0.0285]	W 0.0225 [0.0219, 0.0232]	W 0.0327 [0.0311, 0.0346]	W 0.25 [0.234, 0.252]
Industry	BASE	W 0.19 [0.187, 0.201]	W 14.5 [14.2, 14.7]	W 0.542 [0.528, 0.566]	W 154 [152, 166]	N/M
	R1	L 562 [534, 566]	TO	L 461 [366, 512]	TO	N/M
	R2	W 1.15 [1.13, 1.17]	W 2.96 [2.93, 3.12]	W 0.455 [0.446, 0.463]	W 28 [27.7, 28.3]	N/M
MetaSocket	BASE	W 0.0149 [0.0144, 0.0165]	W 0.0302 [0.027, 0.0308]	W 0.0165 [0.0154, 0.0183]	W 0.0408 [0.0392, 0.0423]	W 0.323 [0.323, 0.324]
	R1	W 0.0136 [0.013, 0.0158]	W 0.0278 [0.0264, 0.0298]	W 0.0147 [0.0146, 0.0153]	W 0.0426 [0.0392, 0.0441]	W 0.443 [0.44, 0.45]
	R2	W 0.0152 [0.0143, 0.0162]	W 0.018 [0.0169, 0.0183]	W 0.0154 [0.0145, 0.0161]	W 0.024 [0.0234, 0.0291]	W 0.29 [0.283, 0.29]
PowerPlant	BASE	W 0.0463 [0.0427, 0.0502]	W 0.487 [0.483, 0.494]	W 0.052 [0.0506, 0.053]	W 1.85 [1.82, 1.87]	W 18 [17.1, 25.7]
	R1	W 0.125 [0.12, 0.147]	W 1.08 [1.04, 1.1]	W 0.152 [0.146, 0.155]	W 5.51 [5.43, 5.58]	W 896 [805, 910]
	R2	W 0.18 [0.178, 0.18]	W 0.251 [0.246, 0.259]	W 0.114 [0.111, 0.117]	W 1.01 [0.991, 1.03]	W 8.25 [7.88, 8.31]
PC Arms=1	BASE	W 0.0741 [0.0678, 0.0754]	W 26 [25.5, 27]	W 0.886 [0.848, 0.912]	W 157 [156, 159]	N/M
	R1	W 0.0793 [0.0748, 0.0811]	W 188 [185, 190]	W 3.15 [3.05, 3.23]	TO	OOM
	R2	W 6.16 [6.06, 6.89]	W 12.2 [12.1, 12.6]	W 1.17 [1.16, 1.19]	W 77.8 [77, 78.2]	N/M
PC Arms=2	BASE	W 2.61 [2.49, 2.95]	TO	TO	TO	OOM
	R1	W 2.6 [2.52, 2.78]	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	W 0.169 [0.164, 0.17]	TO	W 2.26 [2.25, 2.29]	TO	OOM
	R1	W 3.18 [3.12, 3.19]	TO	W 6.74 [6.64, 6.89]	TO	OOM
	R2	W 15.5 [15.5, 15.6]	W 38.7 [38.6, 38.8]	W 4.16 [4.13, 4.19]	W 283 [279, 284]	OOM
Surveillance	BASE	W 2.01 [1.99, 2.03]	TO	W 2.63 [2.41, 2.71]	TO	OOM
	R1	W 18.6 [18.4, 18.8]	TO	W 126 [125, 130]	TO	OOM
	R2	W 37.1 [36.9, 37.5]	TO	W 16.1 [15.3, 16.4]	TO	OOM
Workflow	BASE	W 0.111 [0.108, 0.114]	W 508 [491, 519]	W 0.116 [0.114, 0.119]	TO	OOM
	R1	W 0.325 [0.323, 0.33]	TO	W 0.215 [0.212, 0.221]	TO	OOM
	R2	W 16.1 [15.7, 16.3]	W 23.9 [23.4, 24.5]	W 3.18 [3.11, 3.27]	TO [‡]	OOM

Sampled peak RSS (GiB; five complete trials)

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	W 0.113 [0.112, 0.116]	W 0.122 [0.115, 0.124]	W 0.112 [0.11, 0.113]	W 0.12 [0.116, 0.126]	W 0.144 [0.143, 0.152]
	R1	W 0.111 [0.108, 0.115]	W 0.118 [0.115, 0.12]	W 0.111 [0.11, 0.115]	W 0.124 [0.121, 0.131]	W 0.147 [0.145, 0.154]
	R2	W 0.108 [0.107, 0.111]	W 0.108 [0.107, 0.118]	W 0.11 [0.109, 0.111]	W 0.12 [0.115, 0.123]	W 0.148 [0.141, 0.149]
Industry	BASE	W 0.509 [0.498, 0.529]	W 2.94 [2.69, 3.1]	W 0.703 [0.697, 0.79]	W 5.77 [5.58, 6.4]	N/M
	R1	L 1.48 [1.39, 1.93]	TO	L 1.6 [1.54, 1.6]	TO	N/M
	R2	W 0.881 [0.876, 0.906]	W 1.47 [1.39, 1.6]	W 0.649 [0.623, 0.659]	W 1.98 [1.86, 2.03]	N/M
MetaSocket	BASE	W 0.102 [0.0995, 0.107]	W 0.112 [0.107, 0.113]	W 0.103 [0.102, 0.105]	W 0.116 [0.112, 0.116]	W 0.153 [0.146, 0.159]
	R1	W 0.105 [0.102, 0.106]	W 0.107 [0.107, 0.108]	W 0.104 [0.101, 0.105]	W 0.112 [0.111, 0.115]	W 0.195 [0.19, 0.202]
	R2	W 0.105 [0.103, 0.105]	W 0.104 [0.102, 0.106]	W 0.104 [0.101, 0.105]	W 0.106 [0.105, 0.108]	W 0.141 [0.14, 0.143]
PowerPlant	BASE	W 0.127 [0.124, 0.135]	W 0.436 [0.426, 0.448]	W 0.132 [0.13, 0.137]	W 1.13 [1.08, 1.17]	W 3.25 [3.05, 3.28]
	R1	W 0.181 [0.175, 0.182]	W 0.643 [0.601, 0.654]	W 0.177 [0.171, 0.184]	W 1.57 [1.56, 1.6]	W 8.07 [7.89, 12.5]
	R2	W 0.188 [0.185, 0.193]	W 0.23 [0.228, 0.234]	W 0.177 [0.165, 0.186]	W 0.579 [0.572, 0.587]	W 2.18 [2.16, 2.56]
PC Arms=1	BASE	W 0.145 [0.143, 0.15]	W 5.74 [5.19, 6.01]	W 0.528 [0.474, 0.562]	W 5.37 [5.17, 5.51]	N/M
	R1	W 0.146 [0.144, 0.151]	W 22.6 [22, 23.7]	W 1.07 [1.06, 1.14]	TO	OOM
	R2	W 1.65 [1.52, 1.75]	W 3.08 [2.91, 3.16]	W 0.448 [0.445, 0.458]	W 3.27 [3.18, 3.34]	N/M
PC Arms=2	BASE	W 7.26 [7.2, 7.47]	TO	TO	TO	OOM
	R1	W 7.36 [7.19, 7.54]	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	W 0.794 [0.778, 0.806]	TO	W 0.932 [0.916, 0.942]	TO	OOM
	R1	W 1.47 [1.46, 1.5]	TO	W 1.6 [1.5, 1.62]	TO	OOM
	R2	W 2.86 [2.51, 2.95]	W 5.26 [5.26, 5.29]	W 1.65 [1.62, 1.68]	W 5.47 [5.33, 5.58]	OOM
Surveillance	BASE	W 0.933 [0.928, 0.951]	TO	W 1.4 [1.13, 1.45]	TO	OOM
	R1	W 3 [2.86, 3.02]	TO	W 14.2 [14, 14.5]	TO	OOM
	R2	W 4.4 [4.37, 4.4]	TO	W 2.53 [2.49, 2.55]	TO	OOM
Workflow	BASE	W 1.54 [1.52, 1.63]	W 42.3 [37.6, 45]	W 1.53 [1.5, 1.61]	TO	OOM
	R1	W 1.54 [1.54, 1.55]	TO	W 1.57 [1.51, 1.62]	TO	OOM
	R2	W 2.54 [2.54, 2.64]	W 3.74 [3.71, 3.97]	W 1.61 [1.58, 1.62]	TO [‡]	OOM

Output policy states (first trial)

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	41	42	42	41	86
	R1	39	41	41	41	92
	R2	34	41	39	40	77
Industry	BASE	459	475	476	459	N/M
	R1	–	TO	–	TO	N/M
	R2	441	460	458	438	N/M
MetaSocket	BASE	15	18	18	15	83
	R1	15	18	18	15	79
	R2	15	16	16	15	61
PowerPlant	BASE	59	59	70	59	2434
	R1	32	61	53	65	945
	R2	58	59	46	53	1117
PC Arms=1	BASE	62	78	70	61	N/M
	R1	62	78	70	TO	OOM
	R2	72	75	74	59	N/M
PC Arms=2	BASE	625	TO	TO	TO	OOM
	R1	625	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	206	TO	196	TO	OOM
	R1	136	TO	186	TO	OOM
	R2	203	221	212	204	OOM
Surveillance	BASE	1260	TO	1522	TO	OOM
	R1	376	TO	976	TO	OOM
	R2	982	TO	829	TO	OOM
Workflow	BASE	193	199	199	TO	OOM
	R1	182	TO	194	TO	OOM
	R2	164	198	147	TO [†]	OOM

Output policy transitions (first trial)

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	49	50	50	49	119
	R1	47	49	49	49	117
	R2	42	49	47	48	102
Industry	BASE	1021	1037	1038	1021	N/M
	R1	–	TO	–	TO	N/M
	R2	1008	1022	1020	1000	N/M
MetaSocket	BASE	21	24	24	21	156
	R1	21	24	24	21	143
	R2	21	22	22	21	107
PowerPlant	BASE	78	78	89	78	7796
	R1	51	80	72	84	2303
	R2	77	78	65	72	3148
PC Arms=1	BASE	80	96	88	79	N/M
	R1	80	96	88	TO	OOM
	R2	90	93	92	77	N/M
PC Arms=2	BASE	1360	TO	TO	TO	OOM
	R1	1360	TO	TO	TO	OOM
	R2	TO	TO	TO	TO	OOM
Railcab	BASE	287	TO	276	TO	OOM
	R1	215	TO	266	TO	OOM
	R2	283	300	293	283	OOM
Surveillance	BASE	1638	TO	1900	TO	OOM
	R1	754	TO	1354	TO	OOM
	R2	1360	TO	1207	TO	OOM
Workflow	BASE	243	249	249	TO	OOM
	R1	232	TO	244	TO	OOM
	R2	214	248	197	TO [‡]	OOM

W/L: winning/losing. Seconds: median [min, max] for five valid, consistent completions; ranges in S4. TO: 1,200 s timeout; OOM: out of memory. N/M (capture cap): author-side M9MtsSnapshot instrumentation stops before synthesis at its fixed 250,000-state cap; no evidence of Legacy capability or solver failure. Original operational interruptions are retained in S4. Legacy uses native semantics. Other incomplete groups show statuses, without medians.

Controlled families: first-trial structure

All 132 jobs are retained. These single trials provide no timing medians.

Model	FG lazy	Eager	Update-first	Direct-Full
ctrl_fanout_01	W: 2	W: 2	W: 2	W: 2
ctrl_fanout_02	W: 2	W: 2	W: 2	W: 2
ctrl_fanout_04	W: 2	W: 2	W: 2	W: 2
ctrl_fanout_08	W: 2	W: 2	W: 2	W: 2
ctrl_fanout_16	W: 2	W: 2	W: 2	W: 2
transfer_outcomes_01	W: 2	W: 2	W: 2	W: 2
transfer_outcomes_02	W: 3	W: 3	W: 3	W: 3
transfer_outcomes_04	W: 5	W: 5	W: 5	W: 5
transfer_outcomes_08	W: 9	W: 9	W: 9	W: 9
transfer_outcomes_16	W: 17	W: 17	W: 17	W: 17
uc_events_00	W: 2	W: 2	W: 2	W: 2
uc_events_01	W: 3	W: 3	W: 3	W: 3
uc_events_02	W: 5	W: 5	W: 5	W: 5
uc_events_04	W: 9	W: 9	W: 9	W: 9
uc_events_08	W: 17	W: 17	W: 17	W: 17
uc_outcomes_01	W: 3	W: 3	W: 3	W: 3
uc_outcomes_02	W: 5	W: 5	W: 5	W: 5
uc_outcomes_04	W: 9	W: 9	W: 9	W: 9
uc_outcomes_08	W: 17	W: 17	W: 17	W: 17
requirements_00	W: 2	W: 2	W: 2	W: 2
requirements_01	W: 4	W: 10	W: 4	W: 15
requirements_02	W: 6	W: 26	W: 6	W: 94
requirements_04	W: 10	W: 82	W: 10	W: 2556
requirements_08	W: 18	W: 290	W: 18	W: 1179640
precedence_closure_00	W: 5	W: 11	W: 5	W: 16
precedence_closure_01	W: 5	W: 8	W: 5	W: 12
precedence_closure_02	W: 5	W: 7	W: 5	W: 9
precedence_closure_03	W: 5	W: 6	W: 5	W: 8
precedence_closure_06	W: 5	W: 5	W: 5	W: 5
decision_transfer_complete	W: 3	W: 3	W: 3	W: 3
decision_transfer_partial	L: 3	L: 3	L: 3	L: 3
decision_self_loop_controllable	W: 2	W: 2	W: 2	W: 2
decision_self_loop_uncontrollable	L: 1	L: 1	L: 1	L: 1

Cells show decision and discovered states. Queries and outcomes: `rq4-controlled.csv`.

Travel and hub: completed returns

Travel retains all 48 instances; hub retains 30 seeds under each of three UC profiles. W/TO/OOM are first-trial counts. Five-run timing eligibility is shown separately.

Family	Method	W	TO	OOM	Five-run cells
Travel	FG lazy	20	26	2	20
Travel	Eager	20	26	2	20
Travel	Update-first	20	26	2	20
Travel	Direct-Full	18	28	2	18
Hub	FG lazy	90	0	0	90
Hub	Eager	90	0	0	90
Hub	Update-first	90	0	0	90
Hub	Direct-Full	90	0	0	90

All completed decisions agree. Travel has 960 planned repetition slots: 504 executed trials and 456 explicit resource-failure skips; hub completes all 1,800 trials. `rq4-points.csv` retains all plotted method/instance cells; the two `rq4-* -comparisons.csv` files retain paired ratios and unresolved statuses. Travel lazy/Direct-Full state ratios range from 0.03435 to 0.46154 over 18 completed pairs. The upper endpoint is $N = 1, K = 1$ (12/26 states); all eight $N = 1$ ratios exceed one third. No case is excluded to tighten this range. Lazy and update-first match states, queries and outcomes on all 20 completed Travel instances. Unresolved instances have no measured structure and do not count as equal pairs. Hub timings are solve-only; Travel LTS timings include the internal checks. Ratios are paired within each family, never pooled across these scopes.

Additional timeout evidence. The generated provenance below retains both the original interrupted attempt and its separate same-setting follow-up. The main PC Arms=2/R2 cell continues to report its original 1,200 s TO; the single 3,600 s result is separate evidence.

Authorized RQ3 follow-ups: original and returned records

Workflow/R2 Direct-Full uses the pre-authorized same-setting replacement for its table status (TO[‡]). The original operational interruption remains below. PC Arms=2/R2 retains its original 1,200 s TO; the 3,600 s single trial is a separate sensitivity check. Neither follow-up supplies a median or performance ratio.

Field	Recorded value
Record	PC Arms=2/R2; fg_ducs_otf; original
Campaign / config	rq3 / config.json
Start / end (UTC)	2026-09-15T09:55:34.006646+00:00 / 2026-09-15T10:15:38.375540+00:00
Cap / heap / status	1200 s / 64g / TIMEOUT
Elapsed / peak RSS	1202.9343 s / 31.2834 GiB (process working set)
Original association	rq3/runs/productioncell_arms2__r2__rep01__fg_ducs_otf/meta.json
Record	PC Arms=2/R2; fg_ducs_otf; supplementary
Campaign / config	rq3_supplement_pc_arms2_r2_fg_3600 / config.json
Start / end (UTC)	2026-09-18T06:58:43.395208+00:00 / 2026-09-18T07:58:50.033019+00:00
Cap / heap / status	3600 s / 64g / TIMEOUT
Elapsed / peak RSS	3605.1630 s / 67.1467 GiB (process working set)
Original association	rq3/runs/productioncell_arms2__r2__rep01__fg_ducs_otf/meta.json
Record	Workflow/R2; direct_full; original
Campaign / config	rq3 / config.json
Start / end (UTC)	2026-09-15T00:12:34.878356+00:00 / not recorded
Cap / heap / status	1200 s / 64g / INTERRUPTED_NOT_RETRIED
Elapsed / peak RSS	not recorded / not recorded
Original association	rq3/runs/workflow__r2__rep01__direct_full/meta.json
Record	Workflow/R2; direct_full; supplementary
Campaign / config	rq3_supplement_workflow_r2_direct_full / config.json
Start / end (UTC)	2026-09-18T06:38:27.266316+00:00 / 2026-09-18T06:58:30.415640+00:00
Cap / heap / status	1200 s / 64g / TIMEOUT
Elapsed / peak RSS	1201.6564 s / 13.8951 GiB (process working set)
Original association	rq3/runs/workflow__r2__rep01__direct_full/meta.json

After repetition 1 times out, Workflow repetitions 2–5 have status SKIPPED_AFTER_RESOURCE_FAILURE; all five planned slots are retained in the generated provenance CSV. The PC follow-up planned one trial. Returned artifact digests and saved configuration/plan identities are verified. Input bytes, frozen JAR identity, JVM arguments, and stable host/runtime facts match the original records. The PC follow-up changes only the time cap and repetition count. All use the Xeon W-2265, 24 logical CPUs, Windows 10 build 19045, Temurin 17.0.20.1, and a 64 GiB heap cap. Peak RSS is sampled Windows process working-set memory. Heap occupancy was not recorded; the limiting resource remains unidentified. The recorded elapsed duration includes timeout termination overhead. The interrupted original has no recorded end time or RSS.

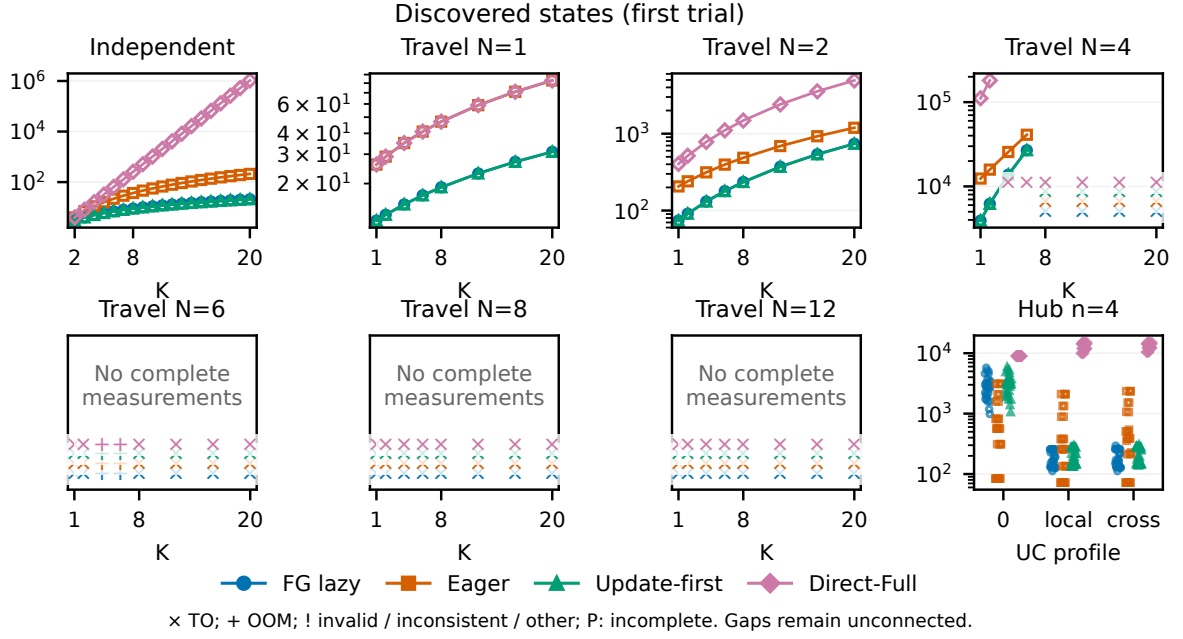


Figure S1: Full scaling-state panels, retaining every tested Travel concurrency level and every hub profile. RQ4 is the archived campaign name; these results appear under RQ3 (Scaling limits) in the main paper. The saved plots label the independent component count as K ; it is n in the main paper and S2, distinct from Travel's choice count K . Censored outcomes have status marks and no numeric ordinate.

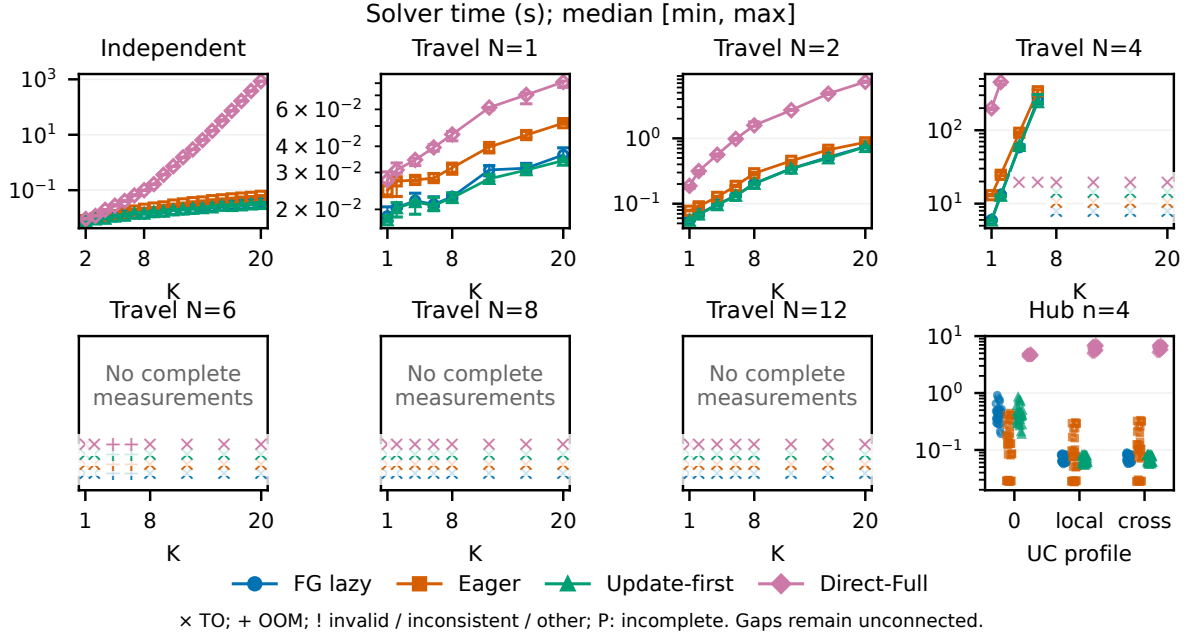


Figure S2: Full scaling-time panels from the archived RQ4 campaign. Medians and ranges require five valid, consistent completions; TO/OOM and other incomplete outcomes remain visible.

Artifact reproduction status. The public anonymous repository contains the full derived source tree, saved RQ1/RQ2 evidence, compiled-monitor exports and Python RS checker, and generated tables/figures. The recorded clean HTTPS clone, build with an empty Maven cache, and RQ1/RQ2 derive-only smoke checks passed; the rederived fields match the saved expectations after path normalization. Tests were compiled but not executed. The README records the environment, elapsed times and anonymized logs in `validation/clean-clone/`, separately from the earlier local build evidence. Third-party dependencies are fetched from official upstream sources; no shaded JAR is distributed. The split raw-output archive is designated for distribution inside the repository tree, rather than a Release asset. The earlier clone check does not validate restoration of the latest expanded raw package; that check and creation of the final `fse27-submission` tag remain publication tasks.

Table S11: All Travel first-trial outcomes. Entries are Lazy/Eager/Update-first/Direct-Full; W = WIN, T = timeout, O = out of memory.

$N \backslash K$	1	2	4	6	8	12	16	20
1	WWWW	WWWW	WWWW	WWWW	WWWW	WWWW	WWWW	WWWW
2	WWWW	WWWW	WWWW	WWWW	WWWW	WWWW	WWWW	WWWW
4	WWWW	WWWW	WWWT	WWWT	TTTT	TTTT	TTTT	TTTT
6	TTTT	TTTT	OOOO	OOOO	TTTT	TTTT	TTTT	TTTT
8	TTTT	TTTT	TTTT	TTTT	TTTT	TTTT	TTTT	TTTT
12	TTTT	TTTT	TTTT	TTTT	TTTT	TTTT	TTTT	TTTT

N counts amenities; K changes UC choices, path length and local state count together. Each outcome comes from the retained first trial under 1,200 s/64 GiB. Resource failures have no median.

S4.1 Qualitative Railcab policy exports

The two qualitative illustrations are separate, single executions on macOS/ARM64 with OpenJDK 17.0.19, using the saved RQ3 campaign JAR, input model, solver properties and 64 GiB heap limit. Only the output options change: `--transition-output full` and `--deployment-output-handoff-bundle.json` retain complete policy bodies. The scripts, exact commands, outputs and extraction checks are in `evidence/railcab_policy_demo` and `evidence/railcab_r1_policy_demo`. Their times are excluded from every performance table. Each was executed once; both return WIN with certificate and Link checks. No independent full-game enumeration or actual runtime experiment was added.

Base exports 115 certificate states, 117 buckets, 118 outcomes, 22 entries, two reached goals and maximum rank 21; R1 exports 44 states, 46 buckets, 46 outcomes, 22 entries, one goal and maximum rank 33. Both have 71 new endpoint states. These structural fields agree with the saved campaign. Full realized-controller transition counts differ from certificate counts and are not substituted for them.

The Base figure begins at certificate state M69, rank 13, reached from old entry O1 after five old stops. All four components are old; every old monitor is inactive, every new monitor is unstarted and Base has no interval monitor. One `reconfigure_MILESTONES` bucket has two outcomes: M15 (new ENDOFTS, rank 11) and M19 (new AC, rank 12). M15 observes uncontrollable `approachingCrossing` to M30 (AC, rank 10), then takes three transfers and seven starts to M20 (rank 0, load new controller state 3). M19 instead takes the three transfers to M16 (rank 9), controllable `brake` to M0 (rank 8), uncontrollable `endOfTS` to M14 (rank 7), then seven starts to M7 (rank 0, load state 1). M19 and M30 have the same milestone location but different `ApproachingCrossingHappen` observer values, so they are distinct states. The expanded descendant fragment has 26 states and 25 edges; no branch is omitted.

Each contracted transfer block uses `ANOTHER_REQUEST`, `REQUEST_ATOMIC`, then `URNS`. Each start block activates, in order, `P_BRAKE_DURING_LASTBRAKE`, `P_BRAKE_IF_SOMETHING_BAD`, `P_CHECK_STATUS_AFTER_APPROACHING_CROSSING`, `P_CHECK_STATUS_BEFORE_LAST_EMERGENCY_BRAKE`, `P_EBRAKE_DURING_ELASTBRAKE`, `P_EBRAKE_IF_SOMETHING_BAD`, and `P_NEW_NOT_CRASH`. Every internal contracted state has exactly one bucket and one outcome. Extraction verifies all 118 rank decreases and both component/new-monitor matches with the fixed endpoint.

R1 retains seven UPD monitors throughout all 44 certificate states, including its goal. Their guards become true after any old stop until the associated new start. Together they forbid `requestEnter`, `brake`, `emergencyBrake`, `idle_c` and `checkCrossingStatus` during the relevant intervals; they do not forbid uncontrollables. The returned policy performs its ordinary controllable steps before its first stop. Milestone transfers occur at LB or NR and each has one outcome. R1 has *zero* multi-outcome buckets. Its branch at M30 instead has two distinct uncontrollable labels, `enterAllowed.0` and `enterAllowed.1`. For entry O1, all 32 descendant states/32 edges are

retained; branches merge before the suffix of five stops, three remaining transfers and seven starts. All $46 \times 7 = 322$ interval-monitor steps match the source DFA, and the goal matches the physical tuple and NEW monitors of new controller state 3. These are checks of the particular returned certificate, not a proof that every winning policy has this order.

Completion-rank coverage. All 25 winning Lazy contracts have five saved worst-completion-rank values, for 125 records. Human-readable and structured metric entries agree in every saved output, and each contract’s rank is identical across its five repetitions. Maximum retained ranks range from 4 to 34, with median 19 across contracts; Railcab Base/R1 have 21/33. The raw summary preserves the other two contracts as LOSS/TO with no winning rank. These are upper bounds on observed game events under the stated execution-progress premises, not downtime measurements or minimal update lengths. The script `evidence/summarize_completion_ranks.py` and all 27 rows in `evidence/completion_ranks.csv` permit rechecking without rerunning synthesis.

S4.2 Auxiliary requirement-boundary inputs

The main paper summarizes all 14 RQ2 decisions. The following supplementary detail reproduces the requirement-boundary comparison. Each input has one old entry. Separation’s endpoints allow only

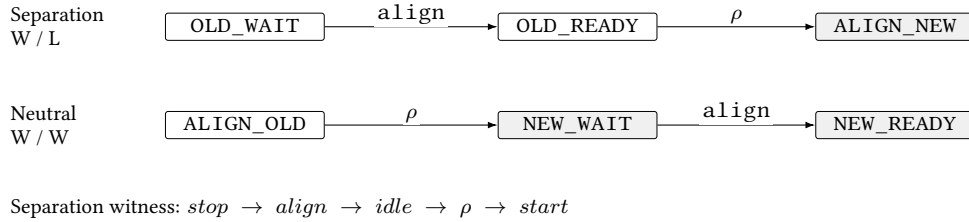


Figure S3: Measured requirement-boundary inputs. Row outcomes are free/block (W: WIN, L: LOSS). All events are controllable; shaded states are new. Each node has an *idle* loop; neutral also has *bad* loops. ρ is the sole transfer. Old/new monitors forbid *align* in separation and *bad* in neutral. An interval monitor permits *start* only after *align* since entry. A requirement block forbids intervening events/transfers between *stop* and *start*. Separation is WIN with free boundaries and LOSS with a block; neutral is WIN in both.

idle; neutral’s old endpoint allows *idle*, and its new controller allows *idle* and the displayed *align*. OLD/NEW testers have one safe state and an error sink; NEW activation requires a false forbidden-event observer. Entry sets *Aligned* false. In separation, *align* sets persistent *Aligned* and a separate observer of the last ordinary event. *idle* clears only the observer; update events preserve both.

The source-derived sequence *stop*, *align*, *idle*, ρ , *start* matches the saved six-state certificate and maximum rank five. Old requirement prevents *align* before *stop*, so a contiguous block cannot complete. Neutral permits ρ , *align* before the block. Its block/free runs return ranks four/five; ranks need not be minimal. These are source-derived witnesses and saved certificate counts, not recovered execution traces.

S4.3 Quiescent-goal functional checks

Three separate one-run fixtures test a safe target match with an enabled uncontrollable event. One old state has a controllable *idle* loop and a transfer to new state n_0 . There are no monitors or dependencies. All new states have controllable *idle* loops. The three cases are:

Case	New uncontrollable edges	Targets	Decision
Persistent	$n_0 \xrightarrow{spin} n_0$	$\{n_0\}$	LOSS
Finite	$n_0 \xrightarrow{step} n_1$	$\{n_0, n_1\}$	WIN
Restricted	$n_0 \xrightarrow{step} n_1$	$\{n_0\}$	LOSS

A target match at n_0 is never a goal while UC is enabled. The finite case reaches quiescent n_1 , with certificate ranks 2, 1, 0 and load into the new controller's state 1. The restricted case cannot return to its only allowed target; its quiescent n_1 is not a goal.

The Python checker independently parses restricted FSP and constructs endpoint products, the entire game, NP and goals, and a least fixed point before comparing with Java. It checks all saved game states and buckets, and for WIN all certificate states, ranks, policy edges and endpoint links. The LOSS result is corroborated by independent full-game closure and Java certificate/exhaustive PASS logs; the CLI does not serialize losing-region membership, so no membership comparison is claimed. The frozen campaign JAR runs once per case under a 512 MiB heap and 120 s cap. Inputs, commands, exports and verification output are retained in `evidence/quiescent_goal_demo/`. These checks are outside the original 92 jobs and every performance statistic.

S4.4 Load-target selector functional checks

Three separate one-run JVM invocations use the frozen campaign JAR and the same one-component input, changing only `loadable_new_states`. The old environment/controller has one idle state. The new environment/controller starts at n_0 , permits a controllable move to n_1 , admits idle at both states and has no return. The sole transfer maps the old state to n_1 . Selecting both endpoint states or only n_1 returns WIN; selecting only n_0 returns LOSS. Although n_0 is reachable in the fixed endpoint, it is unreachable from the update entry.

Both WIN certificates contain two states and one transfer edge, with maximum rank one, and load new controller state 1 at endpoint `new-00000001`. All three complete update-game exports have two reachable states; the LOSS game has no goal. Goal states are terminal in the update export; subsequent idle edges remain in the endpoint bundle. The LOSS CLI exit 6 denotes unrealizability, not an execution failure.

The saved `evidence/load_target_selector_demo/verify_saved.py` parses the fixture's restricted FSP independently, constructs the endpoint products and a Python least fixed point, and compares all exported edges, goals, physical tuples, empty monitor maps, pending transfer, rank and handover. All comparisons pass, as does Java's separate exhaustive construction. There are no monitors, initializer obligations, uncontrollable events or precedence edges. These checks do not validate those features or deployment behavior and are separate from the original 92-job suite and all performance statistics. The directory retains all three inputs, commands, complete game exports, WIN endpoint bundles and verification outputs; no attempt was repeated.

Additional Railcab stop/brake path extraction

The Base and R1 exports share old endpoint entry O1, controller state 1, with identical component and old-monitor states. Their update roots are Base M64 (rank 18) and R1 M28 (rank 27); identifiers are local to each export. Base takes five old stops to M69 (rank 13), then its MILESTONES transfer has both outcomes shown in the main figure. On the lower branch, three further transfers reach M16 (rank 9), whose brake edge reaches M0 (rank 8).

The R1 prefix, without any old stop, consists of:

Source	Event	Target
M28 (27)	requestEnter (C)	M30 (26)
M30 (26)	enterAllowed.0 (UC)	M31 (20)
M31 (20)	lastBrake (UC)	M37 (19)
M37 (19)	reconfigure_MILESTONES (C)	M19 (18)
M19 (18)	brake (C)	M1 (17)
M1 (17)	endOfTS (UC)	M2 (16)
M2 (16)	approachingCrossing (UC)	M13 (15)
M13 (15)	stopOldSpec_P_BRAKE_DURING_LASTBRAKE (C)	M14 (14)

Parentheses give ranks. The final four sources each have one successor. All five old monitors remain active until M13. The stop removes one and changes all seven R1 monitor guards from 0 to 1. Four further stops, three transfers and seven starts reach M3 (rank 0).

The separate *enterAllowed.1* branch completes from the same old entry without brake. This extraction does not establish that braking is necessary for every run. The file `railcab_stop_brake_paths.csv` preserves every edge of the selected Base/R1 entry-to-goal paths (18/22 edges) and all alternative outgoing edges from states on those selected paths. The accompanying extraction script checks path membership, ownership, ranks, common entry and saved UPD-monitor transitions without rerunning synthesis. Original campaign measurements remain unchanged.

Table S12: RQ3 solver medians in seconds: Lazy / Direct-Full for all 27 supplied contracts. Every numeric median uses five completed runs.

Input	Base	R1	R2
GSM	0.0224 / 0.0416	0.0228 / 0.045	0.0201 / 0.0327
Industry	0.19 / 154	562 L / TO	1.15 / 28
MetaSocket	0.0149 / 0.0408	0.0136 / 0.0426	0.0152 / 0.024
PowerPlant	0.0463 / 1.85	0.125 / 5.51	0.18 / 1.01
PC Arms=1	0.0741 / 157	0.0793 / TO	6.16 / 77.8
PC Arms=2	2.61 / TO	2.6 / TO	TO / TO
Railcab	0.169 / TO	3.18 / TO	15.5 / 283
Surveillance	2.01 / TO	18.6 / TO	37.1 / TO
Workflow	0.111 / TO	0.325 / TO	16.1 / TO [†]

Times include checking and exclude JVM startup, frontend and endpoint preparation; three significant digits. Numeric cells are WIN unless L (LOSS); TO is an unfinished first trial, not a median. [†] Workflow/R2 retains its interruption and separate same-setting timeout.

S4.5 Completed-pair exploration counts

The following table retains all completed Lazy/Direct-Full pairs from the same-game campaign. Counts come from the first trial; solver times are medians of five valid runs. Unresolved instances remain in the complete outcome tables above and are not imputed here.

Table S13: All 14 completed application pairs, with exploration and cost.

Input / variant	States (L/DF)	Queries (L/DF)	Solver s (L/DF)
GSM / Base	31 / 75	27 / 658	0.0224 / 0.0416
GSM / R1	34 / 115	44 / 706	0.0228 / 0.0450
GSM / R2	34 / 68	78 / 352	0.0201 / 0.0327
Industry / Base	272 / 282,709	3,336 / 5,058,452	0.190 / 154
Industry / R2	7,717 / 51,096	52,919 / 725,391	1.15 / 28.0
MetaSocket / Base	8 / 82	12 / 289	0.0149 / 0.0408
MetaSocket / R1	8 / 95	12 / 257	0.0136 / 0.0426
MetaSocket / R2	16 / 34	24 / 79	0.0152 / 0.0240
PowerPlant / Base	73 / 8,168	165 / 47,618	0.0463 / 1.85
PowerPlant / R1	759 / 23,686	1,927 / 66,394	0.125 / 5.51
PowerPlant / R2	1,330 / 3,594	3,118 / 13,512	0.180 / 1.01
PC Arms=1 / Base	83 / 407,019	840 / 3,842,690	0.0741 / 157
PC Arms=1 / R2	56,443 / 158,218	354,259 / 1,363,292	6.16 / 77.8
Railcab / R2	97,522 / 317,094	437,383 / 2,842,593	15.5 / 283

L: Lazy; DF: Direct-Full. All 14 completed pairs (WIN/WIN) are shown. States and successor-oracle queries are exact counts from the first trial; solver seconds are medians of five valid runs, including internal checking and excluding JVM/frontend/endpoint time. Times use three significant digits. Unresolved inputs and other methods remain in the full outcome table; this is a completed-pair workload comparison.

S4.6 Extended-budget trials

These single trials extend the budget for previously unresolved conditions; the fixed-budget results remain unchanged. Missing raw is not a failed trial, and a capped single trial supplies no five-run median.

Table S14: Extended-budget single trials (separate from the fixed-budget campaign).

Cell / campaign	Method	GiB	Cap (s)	Status	JVM (s)	Solver (s)	RSS (GiB)	States	JVM ratio
Travel(4,4)/base [ext1]	DF	64	7200	WIN	1.87e+03	1.81e+03	7.36	412688	15.7
Travel(4,6)/base [ext1]	DF	64	7200	WIN	6.99e+03	6.77e+03	12.7	817362	13.4
Travel(4,8)/base [ext1]	Lazy	64	7200	WIN	1.64e+03	756	15.9	47829	–
Travel(4,8)/base [ext1]	Update-first	64	7200	WIN	1.66e+03	718	19.2	47829	–
Industry/r1 [ext2]	DF	64	7200	LOSS	1.7e+03	1.69e+03	2.79	110825	3.01
PC1/r1 [ext2]	DF	64	7200	WIN	2.22e+03	2.22e+03	25.1	3252233	1.68e+03
Travel(4,8)/base [ext3]	DF	64	7200	TO	7.2e+03	–	17.2	–	–
Travel(4,8)/base [ext3]	Eager	64	7200	WIN	2.04e+03	961	17.2	64248	–
Travel(4,12)/base [ext3]	Lazy	64	7200	TO	7.2e+03	–	32	–	–
Travel(6,1)/base [ext3]	Lazy	64	7200	TO	7.2e+03	–	47	–	–
Travel(6,1)/base [ext3]	DF	64	7200	TO	7.2e+03	–	48.6	–	–
Railcab/base [ext4]	DF	200	7200	TO	7.21e+03	–	190	–	–
Surveillance/base [ext4]	DF	200	7200	TO	7.21e+03	–	213	–	–
Workflow/base [ext4]	DF	200	7200	TO	7.21e+03	–	191	–	–
Workflow/r1 [ext4]	DF	200	7200	TO	7.21e+03	–	192	–	–
Railcab/r1 [ext4]	DF	200	7200	TO	7.21e+03	–	212	–	–
PC2/base [ext4]	DF	200	7200	TO	7.21e+03	–	210	–	–
PC2/r1 [ext4]	DF	200	7200	TO	7.21e+03	–	209	–	–
Surveillance/r1 [ext4]	DF	200	7200	OOM	5.27e+03	–	211	–	–
Workflow/r2 [ext4]	DF	200	7200	WIN	1.39e+03	1.38e+03	23.6	1774711	60.1
Surveillance/r2 [ext4]	DF	200	7200	TO	7.21e+03	–	159	–	–
PC2/r2 [ext4]	DF	200	7200	TO	7.21e+03	–	209	–	–
PC2/r2 [ext4]	Lazy	200	7200	TO	7.21e+03	–	187	–	–
Travel(6,1)/base [ext5]	Lazy	200	7200	TO	7.21e+03	–	78.6	–	–
Travel(6,1)/base [ext5]	DF	200	7200	TO	7.21e+03	–	78.9	–	–
Travel(6,4)/base [ext5]	Lazy	200	7200	SKIP	–	–	–	–	–
Travel(6,4)/base [ext5]	DF	200	7200	SKIP	–	–	–	–	–

Each row has one planned trial. WIN/LOSS display SUCCESS/UNREALIZABLE; TO is timeout at the stated cap. NOT_RUN/INCOMPLETE means no completed return; – means unmeasured or ineligible, never zero. SKIP displays SKIPPED_MONOTONE, a scheduling exclusion, not a measured resource failure or an inferred losing decision; reasons are in the CSV. ! marks a provenance/check failure; the raw status is retained. The ratio uses total JVM elapsed time divided by the same cell’s fixed-budget, five-valid-trial Lazy median (1,200 s, 64 GiB), only for matching checked decisions. It differs from the main paper’s solver-time ratios. The CSV separately reports solver-time ratios; timeouts have no ratio. An extended trial is never treated as a five-run median. RSS measures process memory, not heap occupancy. The 5 comparable extended decisions all match fixed-budget Lazy: Direct-Full on Travel (4, 4) and (4, 6), Industry/R1 (LOSS), PC1/R1 and Workflow/R2 (WIN). The 3 Travel (4, 8) decisions agree with one another, but fixed-budget Lazy timed out there; they are not baseline decision matches. JVM elapsed time includes preprocessing and other work outside the solver. For Travel (4, 8), Lazy, Update-first and Eager have JVM/solver times 1,636/756, 1,655/718 and 2,036/961 seconds. Table S14’s JVM ratios compare a single extended trial with a fixed-budget Lazy five-run median; the main paper’s solver ratios use solver time. Separate solver ratios are retained in the CSV.

Industry/R1’s 110,825 states here count the game with goal states terminal. The larger independent-checker count in the Industry/R1 diagnostic above includes reachable states before this truncation, so the enumeration domains differ. Surveillance/R1 has an explicit Java heap space failure. Timeout logs contain no heap-occupancy or GC measurements, so large process RSS alone does not prove a full heap or identify the bottleneck. The 2 Travel (6, 4) skips record the preregistered unmet success prerequisite for (6, 1) under both budgets; they are not observed timeouts or losing decisions.

S4.7 Granularity and construction ablations

E1 merges all transfer events, all old-stop/new-start events, or both on each Base/R1 inherited contract; R2 is excluded because its interval monitors name individual commands. These are contract transformations rather than encodings of the DUCS objective. All 54 transformations finish with 0 changes in decision. The raw physical transfer relations are total. The inherited models omit the explicit post-transfer startup/calibration availability phase used in the constructed examples. These observations explain why those particular mechanisms are absent; they do not constitute a theorem that every total-transfer contract permits simultaneous updating. Requirements can still reject the joint successor, as PC2 below demonstrates.

E4-1 checks the transformations against saved finite fixtures. Transfer/both merging recovers the cell’s losing bulk case. Merging the stop commands with each other and the start commands with each other does not forbid ordinary steps between stop and start: the transformed boundary fixture is therefore distinct from the hand-written contiguous-block restriction. E4-2’s corrected Cell_n rings preserve one circulating workpiece; local and boundary-merged inputs win, transfer/both-merged inputs lose. The original series’ input-validation errors are retained, not treated as synthesis losses. The series are stored in `experiments/witness_20260929/e4_1/` and `experiments/witness_20260929/e4_2_corrected/`.

E5 uniformly restricts each physical transfer domain to the declared initial local state. Of 18 fine/merged pairs, 0 separate, 6 lose both ways, 2 win both ways and 10 remain unresolved. The 25 attempted jobs and 29 deadline-excluded jobs remain separate. Railcab/Base’s fine job timed out; its merged job and the other deadline-excluded jobs were not run. Enabled UC requests at the restricted states can prevent a safe quiescent transfer opportunity. Sources and domain checks are in `experiments/witness_20260929/e5/tables/`.

E2 changes graph construction alone: Full+UC omits controllable buckets at states with enabled UC events, then constructs the reachable graph before solving. The Mac series has an initial capped pass and a longer pass over its timeouts; its final decisions are 16 WIN, 1 LOSS and 10 unresolved. State counts, unlike runtime, can be compared across these saved deterministic constructions. On comparable cells, Full+UC is smaller than Direct-Full in 10 and equal in 6. It is equal in count to Eager in 14 and larger in 2 (GSM/R1 and PowerPlant/R1); equality of the state sets was not checked. OTF without UC pruning is not implemented, so this is an incomplete two-factor comparison. The separate Xeon series uses one fresh trial per cell; Industry/R1 is TO there despite Mac’s LOSS. No Mac runtime is used in the paired Xeon table. Sources: `experiments/ablation_20260928/tables/` and `experiments/ablation_20260928/raw/e2_xeon/`.

Table S15: Construction choices: saved state counts (E2: Mac; other methods: fixed-budget Xeon) and separate Xeon timings. DF = Direct-Full; F+UC = full construction with UC pruning. Time is solve plus internal checking (seconds); F+UC is one trial, Eager a five-run median.

Input	Lazy	Discovered states			Xeon seconds	
		Eager	DF	F+UC	F+UC	Eager
Industry/Base	272	109726	282709	109726	50.455	14.516
PC1/Base	83	407019	407019	407019	158.667	26.016
PC1/R2	56443	158218	158218	158218	77.342	12.155
Railcab/R2	97522	299700	317094	299700	268.563	38.669

Table S16: Selected E2 Xeon timing observations (seconds).

Contract	Full+UC states	Full+UC (1)	Eager (5 med.)	Ratio
Industry/Base	109,726	50.455	14.516	3.48
PC1/Base	407,019	158.667	26.016	6.10
PC1/R2	158,218	77.342	12.155	6.36
Railcab/R2	299,700	268.563	38.669	6.95

Full+UC is one trial; Eager is the median of five valid completed trials. Both timing columns are Xeon observations using the LTS solve-plus-internal-check timer, 64 GiB heap and 1,200 s whole-JVM cap. The displayed Full+UC state counts are identical in the saved Mac and Xeon trials. The full E2 structural comparison uses Mac Full+UC counts and existing Xeon counts for the other methods, retaining each marked budget; it pools no times across hosts. Ratios are descriptive, without repetition-based uncertainty for Full+UC. Equal counts do not establish equal state sets. OTF without UC pruning remains unimplemented.

Table S17: Granularity checks on adapted contracts and controlled variants.

Series	Comparisons/trials	WIN	LOSS	TO/NR
E1 transformations	54	51	3	0
E4 fixtures	24	19	5	0
E4 corrected rings	45	27	18	0
E5 scheduled trials	54	4	12	9/29

E1 changes no decision in 54 comparisons; 44 discovered-state counts decrease and 10 are equal. The 22 original physical transfer relations are total, which alone does not prove safety or completion. E4 rings cover $n = 2, \dots, 10$; the initial series with 30 input errors is retained separately. Boundary batching is not the handmade contiguous-block restriction. E5 has 18 pairs: no witness, six both-LOSS, two both-WIN, ten unresolved; NR denotes the deadline exclusion.

Table S18: E6 registered trials by saved series, with hosts and additional budgets kept separate.

Series	Trials	WIN	LOSS	TO
rolling/v1	100	50	50	0
canary/v1	60	30	30	0
policy/v2	6	3	3	0
db_rolling/v2	8	2	6	0
rolling_audit/v1	8	4	4	0
threads/v1	80	40	40	0
rolling_scale/v1	80	40	40	0
canary_controls/v1	48	24	24	0
pc2_rolling/v2	3	1	0	2
pc2_rolling/extended_7200	2	0	0	2
pc2_rolling/xeon_20260929_2344	3	1	0	2

The 55 main comparisons are 33 witnesses, 11 both-LOSS, 10 both-WIN and one unresolved PC2 comparison. Rolling scale (20 pairs), Canary controls (12 pairs), and E4 references (9 pairs) are separate. These deliberately constructed cases do not estimate prevalence. PC2 measured TO remains TO despite its separately checked safety obstruction. Audit boundary merging remains WIN; Threads supplies ten both-WIN and ten both-LOSS controls. All new timings are single trials, not medians.

S4.8 Constructed operational mechanisms and controls

The E6 input generators construct finite abstractions motivated by documented staged-replacement practices; they are not deployment measurements or replications of production incidents. The final index contains 398 terminal trials (195 WIN, 197 LOSS, 6 TO), including 395 Mac and 3 Xeon trials. The main comparison population is 55 pairs; scale extensions, assumption controls, E4 references and follow-up budgets do not increase that denominator. Full model definitions, generated inputs, prior failed versions, independent checks and primary-source scope are retained under `experiments/witness_20260929/e6/`.

Rolling models a fixed number of replicas and a readiness floor. Updating a batch introduces a finite startup gap; the checked threshold concerns this abstraction, not arbitrary asynchronous deployment. Canary adds observed set-valued healthy/broken outcomes and recovery: controls with only healthy outcomes win even when merged, while removing recovery loses under both granularities. Policy’s opposing requirement-boundary orders separate individual from globally merged boundaries. DB-Rolling models secondary-first replacement with a retained availability requirement; the no-slack control loses under both granularities. Rolling+Audit has only one old-stop/new-start pair, so boundary merging remains WIN; transfer merging loses. Its displayed six-step path comes from the Full policy, not Lazy’s returned maximum rank. Threads models backpressure versus saturation; the former wins both ways and the latter loses both ways. These null controls prevent reading every staged update as a granularity separation.

PC2-Rolling: measured and checked facts. The fine contract returns WIN on the two-arm physical plant with a calibration phase. Its exported game has 6639 discovered states, 381 policy states and 134128 queries; the Xeon counters agree with the saved Mac result. A separate enumeration of the old closed-loop product checks that both raw transfer domains contain every reachable old state. Their product nevertheless puts both arms in calibration, immediately violating the interval availability requirement. Ordinary old-product moves preserve the enumerated invariant, and any completing joint-transfer policy must eventually take that violating transfer; this gives a structural impossibility argument independently of the unresolved solver. The fine Lazy Xeon trial completes in 99.024 total JVM seconds; the merged Lazy and fine Direct-Full trials both time out at 7200 seconds with a 200 GiB heap. Earlier Mac timeouts remain separate records. Raw exit codes and runtime class hashes are unavailable for these Xeon exports and are not reconstructed. The saved invariant check is not a claim that the larger game was independently enumerated in full.

S4.9 Separate Legacy enlarged-heap reference

Table S19: Separate Legacy extended-budget resource observations: one Xeon trial per condition, 200 GiB heap and 3,600 s cap.

Contract	Display	Peak states	Peak edges	JVM s	RSS GiB	Source
Railcab/BASE	N/M (profile)	5,320,658	80,379,005	1511.9	193.94	a
Workflow/BASE	OOM	–	–	3541.4	207.59	b
Surveillance/BASE	OOM	–	–	2668.6	207.71	c
PC2/BASE	OOM	–	–	1550.6	207.80	d
Railcab/R1	OOM	–	–	3209.7	207.71	e
Workflow/R1	OOM	–	–	3396.3	207.64	f
Surveillance/R1	OOM	–	–	1888.1	207.73	g
PC2/R1	OOM	–	–	1696.5	207.75	h
Railcab/R2	N/M (profile)	5,320,658	80,379,005	1646.1	200.00	i
Workflow/R2	OOM	–	–	2593.9	207.72	j
Surveillance/R2	OOM	–	–	2067.3	207.79	k
PC2/R2	OOM	–	–	774.1	207.76	l
PC1/R1	OOM	–	–	2240.0	209.41	m
PC1/BASE	N/M (profile)	1,814,373	25,589,560	417.2	72.76	n
PC1/R2	N/M (profile)	1,814,373	25,589,560	436.2	80.83	o
Industry/BASE	N/M (profile)	683,129	8,136,485	141.5	24.81	p
Industry/R1	N/M (profile)	2,888,907	34,072,018	713.7	93.66	q
Industry/R2	N/M (profile)	683,129	8,136,485	145.6	25.95	r

N/M (profile) preserves raw CRASH: all seven records report an `IllegalArgumentException` at `M9MtsSnapshot.requireResourceCensus` while capturing the safety meta-environment, before final controller synthesis. The snapshot profile limits include 250,000 states; this is separate from the heap/time budget. State/edge figures are the recorded peak intermediate state space, not a controller or completed solution. All eleven OOM records report Java heap space. RSS is sampled Windows working set, not heap occupancy. Failure runtimes are whole-JVM observations, not successful-solver timings or medians. No decision or general tool limitation is inferred. Text macros round recorded state counts (millions) and OOM RSS (GiB) to one decimal. Sources below are relative to `experiments/rq3_xeon/raw/ext6_legacy_heap200/`; each directory supplies `meta.json`, `output.txt` and `stderr.txt`.

- a runs/railcab__base__rep01__legacy_ducs/
- b runs/workflow__base__rep01__legacy_ducs/
- c runs/surveillance__base__rep01__legacy_ducs/
- d runs/productioncell_arms2__base__rep01__legacy_ducs/
- e runs/railcab__r1__rep01__legacy_ducs/
- f runs/workflow__r1__rep01__legacy_ducs/
- g runs/surveillance__r1__rep01__legacy_ducs/
- h runs/productioncell_arms2__r1__rep01__legacy_ducs/
- i runs/railcab__r2__rep01__legacy_ducs/
- j runs/workflow__r2__rep01__legacy_ducs/
- k runs/surveillance__r2__rep01__legacy_ducs/
- l runs/productioncell_arms2__r2__rep01__legacy_ducs/
- m runs/productioncell_arms1__r1__rep01__legacy_ducs/
- n runs/productioncell_arms1__base__rep01__legacy_ducs/
- o runs/productioncell_arms1__r2__rep01__legacy_ducs/
- p runs/industry__base__rep01__legacy_ducs/
- q runs/industry__r1__rep01__legacy_ducs/
- r runs/industry__r2__rep01__legacy_ducs/

S4.10 Separate Eager enlarged-heap trials

Each of the 10 conditions has one Eager trial on the same Xeon/JAR as the fixed-budget campaign, with a 200 GiB Java heap and a 7,200 s process-time cap; these are single observations, not timing medians, and no Mac times enter this comparison. All 4 decisions match fixed-budget Lazy; the state ratio is Eager’s `states_discovered` divided by Lazy’s `states_discovered` (structure repetition 1, equal over all five valid baseline repetitions), and is reported only when both decide. The 5 timeouts and 1 Java-heap OOM have no completed state/query/solver-time record or ratio; their JVM times include termination overhead, and sampled working-set RSS is process memory rather than heap occupancy. Counts describe the states discovered by the UC-pruned Eager search, which treats unsafe/goal states as terminal; they do not equate its state set with E2 or the unpruned Direct-Full game.

Table S20: Eager enlarged-heap trials (ext7). W = WIN; L = LOSS; TO = timeout. Times are seconds and RSS is GiB; each timing is one trial.

Condition	Status	States	Queries	Solver	JVM	RSS	Lazy states	Ratio	Src.
Industry/R1	L	94,046	635,100	1,331.5	1,333.4	2.51	38,942	2.4	a
Railcab/Base	W	11,659,723	115,706,744	2,557.3	2,570.3	125.54	171	68,186	b
Workflow/R1	W	26,594,216	111,677,312	2,406.6	2,428.9	184.94	1,098	24,221	c
Surv./Base	TO	–	–	–	7,205.1	211.94	5,319	–	d
Railcab/R1	TO	–	–	–	7,205.2	209.62	13,339	–	e
Surv./R1	TO	–	–	–	7,205.1	212.13	96,301	–	f
Surv./R2	W	10,841,519	55,401,437	1,575.3	1,591.7	134.43	181,116	59.9	g
PC2/Base	TO	–	–	–	7,205.2	209.66	2,502	–	h
PC2/R1	OOM	–	–	–	2,981.3	209.06	2,433	–	i
PC2/R2	TO	–	–	–	7,205.2	208.83	–	–	j

Solver time includes internal checking; JVM includes preparation and process shutdown. The status/count/timing fields are cross-checked against each job’s `meta.json`, `output.txt` and `stderr.txt`. Source directories below are relative to `experiments/rq3_xeon/raw/ext7_eager_heap200/`; baseline counts come from `paper/build/generated/rq3-cells.csv` and agree with all five corresponding raw repetitions.

- a `runs/industry__r1__rep01__fg_ducs_otf_eager_controllable/`
- b `runs/railcab__base__rep01__fg_ducs_otf_eager_controllable/`
- c `runs/workflow__r1__rep01__fg_ducs_otf_eager_controllable/`
- d `runs/surveillance__base__rep01__fg_ducs_otf_eager_controllable/`
- e `runs/railcab__r1__rep01__fg_ducs_otf_eager_controllable/`
- f `runs/surveillance__r1__rep01__fg_ducs_otf_eager_controllable/`
- g `runs/surveillance__r2__rep01__fg_ducs_otf_eager_controllable/`
- h `runs/productioncell_arms2__base__rep01__fg_ducs_otf_eager_controllable/`
- i `runs/productioncell_arms2__r1__rep01__fg_ducs_otf_eager_controllable/`
- j `runs/productioncell_arms2__r2__rep01__fg_ducs_otf_eager_controllable/`

S5 Expanded related-work comparison

Table S21: Expanded primary-source comparison of verified native contracts. C/R: component/requirement granularity; T: transfer; Res/P: residual obligations/precedence; Roots/O: initial states/outcomes. Publication details are in the main bibliography; section/definition numbers locate the inspected contracts.

Work	Native C/R; T; Res/P	Roots/O; progress premises	Artifact and construction
DUCS (2016, 2020)	Global environment/specification switch; set-valued R ; steps inside E' ; transition formula T constrains update order (Sec. 4.2)	All old states via total H ; all outcomes; eventual update events after hotSwap; legal and deadlock-free (Defs. 4.1–4.3)	Update controller; FLTL control reduction and MTSA synthesis
GR(1) bridge (2022)	Two specifications and switching assertion; old safety guarantees before switch, new specification afterward	All prefixes meeting new environment safety; uniformly bounded switch (Def. 3.1); subsequent GR(1) liveness assumes justice	Symbolic reachability bridge; winning set W or current-state early stop
Live Synthesis (2022)	Old/new LTL and trace or transition system; residual $\Psi(\eta, \varphi)$; old obligations finitely satisfied	Universal variant: all old finite traces and new traces; reduction assumes one eventual update (Thm. 6.4)	Obligation monitor and LTL synthesis; LiveLTL language inclusion
DES state attraction (2015)	Common deterministic plant; request/startup sets; task language K and recognizer	All request states; UC preserved; weakly invariant startup set; bounded first completion (Def. 2, Thm. 1)	Attractor supervisor; plant/recognizer product and state-attraction algorithm
OTF-DCS (2023)	Component automata, marked/illegal states and control partition	All UC enabled; designated disturbances; eager director (Def. 3); every controlled finite trace has a marked extension (Def. 4)	Safe nonblocking director; heuristic OTF composition
Tripakis–Altisen (1999)	Finite game graph, controllable/UC edges, safe/target sets	UC-successor closure; controllable path to target from each node (Sec. 2.1); environment may move first	Strategy subgraph; fully OTF for untimed graphs
Strong planning (1998)	Action/fluent domain, nondeterministic effects and goal formula	All initial states and outcomes; strong completion; fair iterative extension separate (pp. 40–43)	OBDD state/action plan; universal outcome preimages; initial-set early stop
Dynamic Feature (2024)	Feature come/go, component reinitialization, simultaneous changes; behavioral/configuration constraints	Mixed controllability; UC preserved; every reachable controlled state has a jointly marked continuation (Sec. 7)	CIF BDD synthesis; maximal safe controllable nonblocking supervisor
Network updates (2015)	Per-switch replacement; searched update order; packet-flush waits	All single-packet traces satisfy LTL; failure freedom, fair forwarding, bounded processing (Sec. 3.1)	Sequence; DFS and incremental checking; simple/careful completeness